EXPERIMENT 1: TITANIC DATASET PREPROCESSING AND VISUALIZATION

AIM:

To perform data preprocessing and exploratory data analysis on the Titanic dataset by inspecting data, handling missing values and duplicates, encoding categorical data, scaling numerical data, and visualizing feature relationships.

ALGORITHM:

1. Import the required Python libraries: Pandas, Seaborn, Matplotlib, and scikit-learn preprocessing tools.
2. Load the Titanic CSV dataset into a Pandas DataFrame and inspect the first rows, data types, and missing values.
3. Fill missing Cabin values with “Unknown” and remove duplicate records.
4. Encode the Sex column using LabelEncoder.
5. Standardize the Fare column using StandardScaler.
6. Fill missing Age values using forward fill and backward fill.
7. Plot selected features (Pclass, Sex, Age, SibSp) using a pair plot.
8. Calculate and display the correlation heatmap for Pclass, Age, SibSp, Parch, and Fare.

Step 1: Load the Titanic Dataset and Convert it into a DataFrame

```
[ ]
#EXP_NO_1
#SANJAY_S_241801246
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler

from google.colab import drive
drive.mount('/content/drive')

file_path = "/content/drive/MyDrive/PODS/Titanic-Dataset.csv"
df = pd.read_csv(file_path)
```

Step 2: Explore and Understand the Dataset

Display the first few rows

```
]
#EXP_NO_1
#SANJAY_S_241801246
df.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

Get information about column data types and missing values

```
#EXP_NO_1
#SANJAY_S_241801246
df.info()
print("\nMissing Values:")
print(df.isnull().sum())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   PassengerId  891 non-null    int64  
 1   Survived     891 non-null    int64  
 2   Pclass       891 non-null    int64  
 3   Name         891 non-null    object 
 4   Sex          891 non-null    object 
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64  
 7   Parch        891 non-null    int64  
 8   Ticket       891 non-null    object 
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object 
 11  Embarked     889 non-null    object 
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB

Missing Values:
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64
```

Step 3: Do Forward Fill and Backward Fill on 'Age'

```
[ ]
#EXP_NO_1
#SANJAY_S_241801246
df["Age"] = df["Age"].ffill()
df["Age"] = df["Age"].bfill()
print(df["Age"].isnull().sum())
```

```
0
```

Step 4: Fill Missing Values in 'Cabin' with "unknown" (Limit = 5)

```
[ ]
#EXP_NO_1
#SANJAY_S_241801246
df["Cabin"] = df["Cabin"].fillna("unknown", limit=5)
print(df["Cabin"].isnull().sum())
```

```
682
```

Step 5: Remove Duplicate Records (if any)

```
[ ]
#EXP_NO_1
#SANJAY_S_241801246
print("Duplicate Records:", df.duplicated().sum())
df = df.drop_duplicates()
print("Dataset Shape:", df.shape)
```

```
Duplicate Records: 0
Dataset Shape: (891, 12)
```

Step 6: Encode Categorical Column 'Sex' using LabelEncoder

```
#EXP_NO_1
#SANJAY_S_241801246
encoder = LabelEncoder()
df["Sex"] = encoder.fit_transform(df["Sex"])
df[["Sex"]].head()
```

	Sex
0	1
1	0
2	0
3	0
4	1

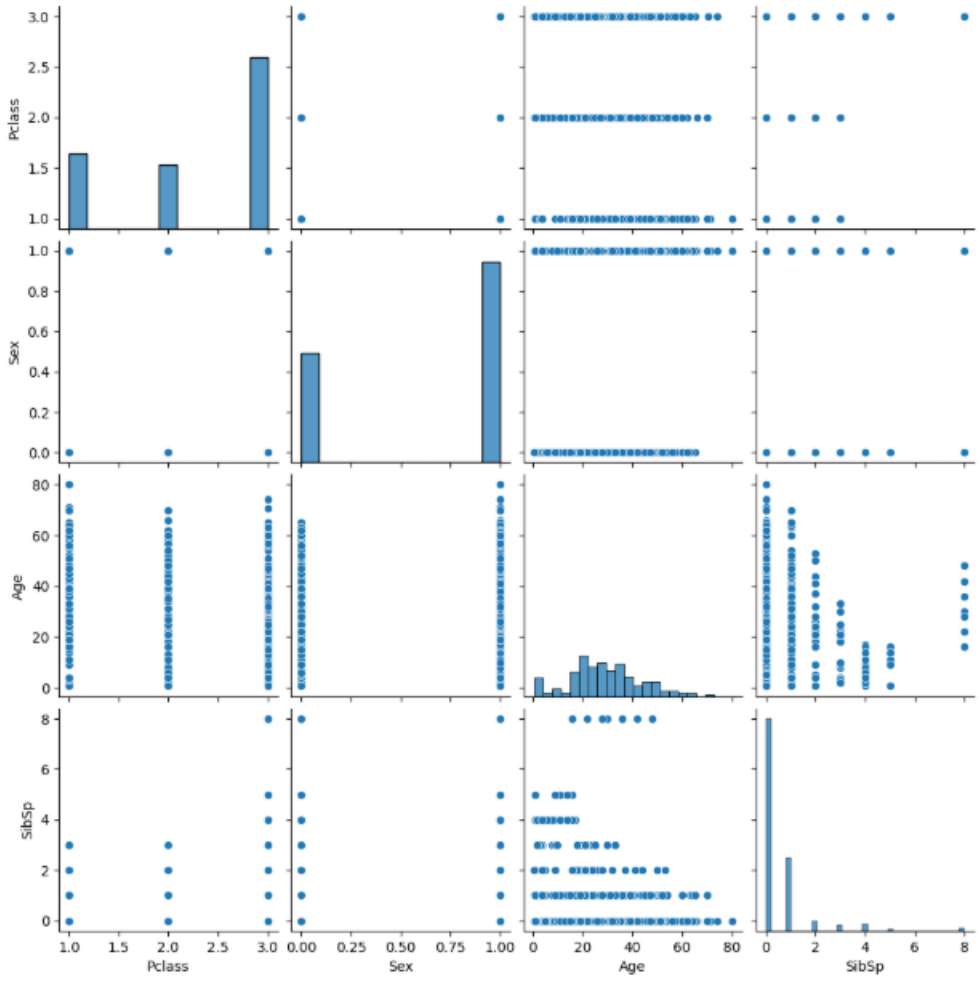
Step 7: Scale Numerical Feature 'Fare' using StandardScaler

```
#EXP_NO_1
#SANJAY_S_241801246
scaler = StandardScaler()
df["Fare"] = scaler.fit_transform(df[["Fare"]])
df[["Fare"]].head()
```

	Fare
0	-0.502445
1	0.786845
2	-0.488854
3	0.420730
4	-0.486337

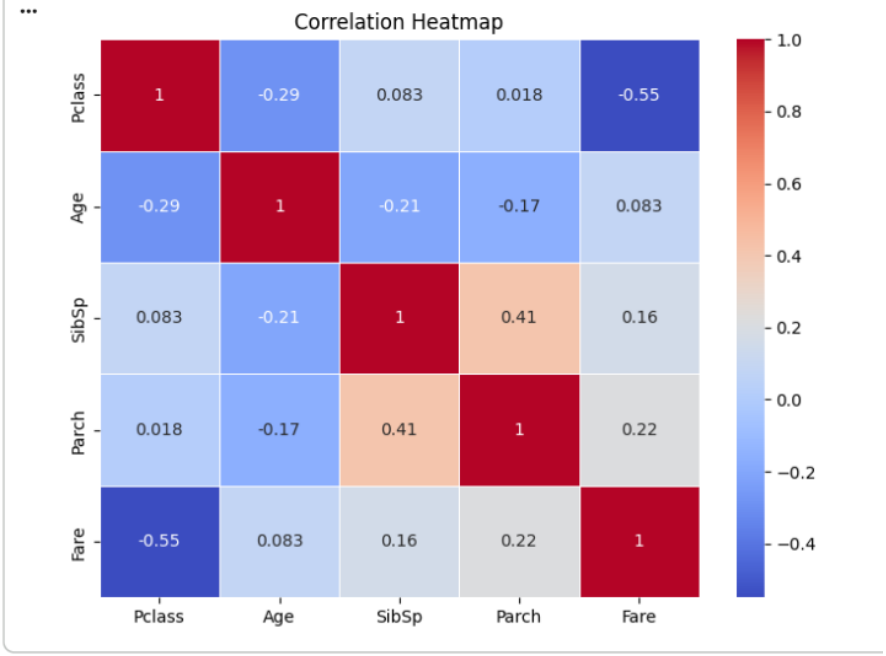
Step 8: Pair Plot of Selected Features ('Pclass', 'Sex', 'Age', 'SibSp')

```
#EXP_NO_1
#SANJAY_S_241801246
sns.pairplot(df[['Pclass', 'Sex', 'Age', 'SibSp']])
plt.show()
```



Step 9: Display the Correlation Heatmap ('Pclass', 'Age', 'SibSp', 'Parch', 'Fare')

```
#EXP_NO_1
#SANJAY_S_241801246
plt.figure(figsize=(8,6))
corr = df[['Pclass', 'Age', 'SibSp', 'Parch', 'Fare']].corr()
sns.heatmap(corr, annot=True, cmap="coolwarm", linewidths=0.5)
plt.title("Correlation Heatmap")
plt.show()
```



RESULT:

The Titanic dataset was loaded into a DataFrame and explored. Missing values were handled, duplicate records were removed, the Sex column was label-encoded, the Fare column was standardized, and visualizations (pair plot and correlation heatmap) were generated to examine feature distributions and relationships.

EXPERIMENT 2: TITANIC DATASET ANALYSIS

AIM:

To analyze the Titanic dataset using Python by inspecting dataset information and summary statistics, handling missing values, visualizing passenger-class distribution, filtering passengers based on specified conditions, and splitting the data into training and testing sets.

ALGORITHM:

1. Import the required libraries: Pandas, Seaborn, Matplotlib, SimpleImputer, and train_test_split.
2. Load the Titanic CSV dataset into a Pandas DataFrame and display the first five rows.
3. Display the dataset shape, information, and summary statistics.
4. Fill missing values in the Age column using SimpleImputer with the mean strategy.
5. Replace missing values in the Cabin column with “Unknown”.
6. Fill missing values in the Embarked column using its mode.
7. Plot the passenger class distribution using a count plot.
8. Display female passengers who survived.
9. Display third-class passengers younger than 18 years.
10. Display first-class passengers older than 40 years.
11. Display surviving passengers in the specified category.
12. Display male passengers whose fare is greater than 100.
13. Display passengers who embarked from C and belonged to passenger class 2.
14. Display passengers with SibSp greater than 2.
15. Display non-surviving passengers who had no siblings/spouses and no parents/children aboard.
16. Display the five oldest surviving passengers.
17. Display passengers who paid zero fare.
18. Split the dataset into training and testing sets with test_size=0.2 and random_state=42.

Step 1: Load the Titanic Dataset and Convert it into a DataFrame

```
#EXP_NO_2
#Sanjay_S_241801246
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split

from google.colab import drive
drive.mount('/content/drive')

file_path = "/content/drive/MyDrive/PODS/Titanic-Dataset.csv"
df = pd.read_csv(file_path)
```

```
Mounted at /content/drive
```

Step 1: Load the Titanic Dataset and Convert it into a DataFrame

```
#EXP_NO_2
#Sanjay_S_241801246
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split

from google.colab import drive
drive.mount('/content/drive')

file_path = "/content/drive/MyDrive/PODS/Titanic-Dataset.csv"
df = pd.read_csv(file_path)
```

```
Mounted at /content/drive
```

Step 2: Understand the Data (Shape, Info, Describe)

```
#EXP_NO_2
#Sanjay_S_241801246

df.shape
df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
 4   Sex          891 non-null    object
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object
 11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

Step 3: Fill the Missing Values in Age using SimpleImputer

```
#EXP_NO_2
#Sanjay_S_241801246
imputer = SimpleImputer(strategy='mean')
df["Age"] = imputer.fit_transform(df[["Age"]])
print(df["Age"].isnull().sum())
```

```
0
```

Step 4: Fill Missing Cabin with "Unknown"

```
#EXP_NO_2
#Sanjay_S_241801246
df["Cabin"] = df["Cabin"].fillna("Unknown")
print(df["Cabin"].isnull().sum())
```

```
0
```

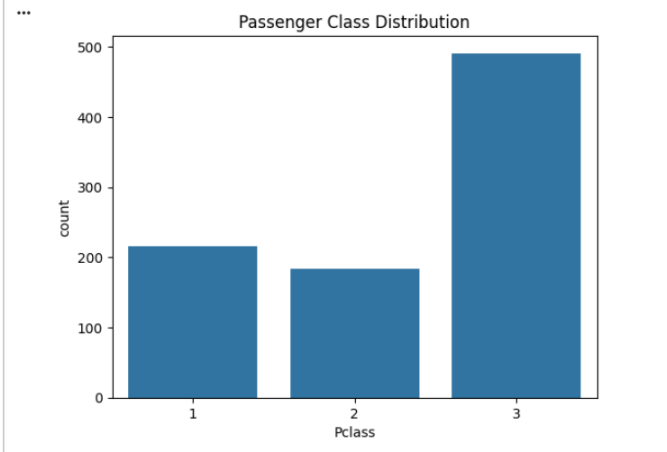
Step 5: Fill Missing Embarked with the Mode

```
#EXP_NO_2
#Sanjay_S_241801246
mode_value = df["Embarked"].mode()[0]
df["Embarked"] = df["Embarked"].fillna(mode_value)
print(df["Embarked"].isnull().sum())
```

```
0
```

Step 6: Analyse the Passenger Class Feature using Countplot

```
#EXP_NO_2
#Sanjay_S_241801246
sns.countplot(x="Pclass", data=df)
plt.title("Passenger Class Distribution")
plt.show()
```



Step 7: List the Names of Passengers who are Female and Survived

```
#EXP_NO_2
#Sanjay_S_241801246
female_survived = df[(df["Sex"] == "female") & (df["Survived"] == 1)]
female_survived[["Name"]]
```

	Name
1	Cumings, Mrs. John Bradley (Florence Briggs Th...
2	Heikkinen, Miss. Laina
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)
8	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)
9	Nasser, Mrs. Nicholas (Adele Achem)
...	...
874	Abelson, Mrs. Samuel (Hannah Wizosky)
875	Najib, Miss. Adele Kiamie "Jane"
879	Potter, Mrs. Thomas Jr (Lily Alexenia Wilson)
880	Shelley, Mrs. William (Imanita Parrish Hall)
887	Graham, Miss. Margaret Edith

233 rows × 1 columns

Step 8: Find all Passengers who are in 3rd Class and Under the Age of 18

```
#EXP_NO_2
#Sanjay_S_241801246
third_class = df[(df["Pclass"] == 3) & (df["Age"] < 18)]
third_class[["Name", "Age", "Pclass"]]
```

	Name	Age	Pclass
7	Palsson, Master. Gosta Leonard	2.0	3
10	Sandstrom, Miss. Marguerite Rut	4.0	3
14	Vestrom, Miss. Hulda Amanda Adolfina	14.0	3
16	Rice, Master. Eugene	2.0	3
22	McGowan, Miss. Anna "Annie"	15.0	3
...	...	...	...
844	Culumovic, Mr. Jeso	17.0	3
850	Andersson, Master. Sigvard Harald Elias	4.0	3
852	Boulos, Miss. Nourelain	9.0	3
869	Johnson, Master. Harold Theodor	4.0	3
875	Najib, Miss. Adele Kiamie "Jane"	15.0	3

78 rows × 3 columns

Step 9: Find the List of Passengers who are in Class 1 and Older than 40

```
#EXP_NO_2
#Sanjay_S_241801246
class1 = df[(df["Pclass"] == 1) & (df["Age"] > 40)]
class1[["Name", "Age", "Pclass"]]
```

	Name	Age	Pclass
6	McCarthy, Mr. Timothy J	54.0	1
11	Bonnell, Miss. Elizabeth	58.0	1
35	Holverson, Mr. Alexander Oskar	42.0	1
52	Harper, Mrs. Henry Sleeper (Myna Haxtun)	49.0	1
54	Ostby, Mr. Engelhart Cornelius	65.0	1
...	...	...	...
856	Wick, Mrs. George Dennick (Mary Hitchcock)	45.0	1
857	Daly, Mr. Peter Denis	51.0	1
862	Swift, Mrs. Frederick Joel (Margaret Welles Ba...	48.0	1
871	Beckwith, Mrs. Richard Leonard (Sallie Monypeny)	47.0	1
879	Potter, Mrs. Thomas Jr (Lily Alexenia Wilson)	56.0	1

76 rows × 3 columns

Step 10: Find the List of Passengers Survived in the Above Category

```
#EXP_NO_2
#Sanjay_S_241801246
survived_class1 = df[(df["Pclass"] == 1) &
                     (df["Age"] > 40) &
                     (df["Survived"] == 1)]
survived_class1[["Name", "Age", "Pclass", "Survived"]]
```

299	Baxter, Mrs. James (Helene DeLaudeniere Chaput)	50.0	1	1
337	Burns, Miss. Elizabeth Margaret	41.0	1	1
366	Warren, Mrs. Frank Manley (Anna Sophia Atkinson)	60.0	1	1
380	Bidois, Miss. Rosalie	42.0	1	1
449	Peuchen, Major. Arthur Godfrey	52.0	1	1
453	Goldenberg, Mr. Samuel L	49.0	1	1
460	Anderson, Mr. Harry	48.0	1	1
496	Eustis, Miss. Elizabeth Mussey	54.0	1	1
513	Rothschild, Mrs. Martin (Elizabeth L. Barrett)	54.0	1	1
523	Hippach, Mrs. Louis Albert (Ida Sophia Fischer)	44.0	1	1
556	Duff Gordon, Lady. (Lucille Christiana Sutherl...	48.0	1	1
571	Appleton, Mrs. Edward Dale (Charlotte Lamson)	53.0	1	1
587	Frolicher-Stehli, Mr. Maxmillian	60.0	1	1
591	Stephenson, Mrs. Walter Bertram (Martha Eustis)	52.0	1	1
599	Duff Gordon, Sir. Cosmo Edmund ("Mr Morgan")	49.0	1	1
621	Kimball, Mr. Edwin Nelson Jr	42.0	1	1

Step 11: Which Male Passengers Paid a Fare Greater than 100?

```
#EXP_NO_2
#Sanjay_S_241801246
male_fare = df[(df["Sex"] == "male") & (df["Fare"] > 100)]
male_fare[["Name", "Fare"]]
```

	Name	Fare
27	Fortune, Mr. Charles Alexander	263.0000
118	Baxter, Mr. Quigg Edmond	247.5208
305	Allison, Master. Hudson Trevor	151.5500
332	Graham, Mr. George Edward	153.4625
373	Ringhini, Mr. Sante	135.6333
377	Widener, Mr. Harry Elkins	211.5000
390	Carter, Mr. William Ernest	120.0000
438	Fortune, Mr. Mark	263.0000
505	Penasco y Castellana, Mr. Victor de Satode	108.9000
527	Farthing, Mr. John	221.7792
544	Douglas, Mr. Walter Donald	106.4250

Step 12: List Passengers who Embarked from Port 'C' and were in Class 2

```
#EXP_NO_2
#Sanjay_S_241801246
embarked_c = df[(df["Embarked"] == "C") & (df["Pclass"] == 2)]
embarked_c[["Name", "Embarked", "Pclass"]]
```

	Name	Embarked	Pclass
9	Nasser, Mrs. Nicholas (Adele Achem)	C	2
43	Laroche, Miss. Simonne Marie Anne Andree	C	2
122	Nasser, Mr. Nicholas	C	2
135	Richard, Mr. Emile	C	2
181	Pernot, Mr. Rene	C	2
292	Levy, Mr. Rene Jacques	C	2
308	Abelson, Mr. Samuel	C	2
361	del Carlo, Mr. Sebastiano	C	2
389	Lehmann, Miss. Bertha	C	2
473	Jerwan, Mrs. Amin S (Marie Marthe Thuillard)	C	2
547	Padro y Manent, Mr. Julian	C	2
608	Laroche, Mrs. Joseph (Juliette Marie Louise La...	C	2
685	Laroche, Mr. Joseph Philippe Lemercier	C	2
817	Mallet, Mr. Albert	C	2

Step 13: Show all Passengers who had More than 2 Siblings/Spouses Aboard (SibSp > 2)

```
#EXP_NO_2
#Sanjay_S_241801246
sibsp = df[df["SibSp"] > 2]
sibsp[["Name", "SibSp"]]
```

171	Rice, Master. Arthur	4
176	Lefebre, Master. Henry Forbes	3
180	Sage, Miss. Constance Gladys	8
182	Asplund, Master. Clarence Gustaf Hugo	4
201	Sage, Mr. Frederick	8
229	Lefebre, Miss. Mathilde	3
233	Asplund, Miss. Lillian Gertrud	4
261	Asplund, Master. Edvin Rojj Felix	4
266	Panula, Mr. Ernesti Arvid	4
278	Rice, Master. Eric	4
324	Sage, Mr. George John Jr	8
341	Fortune, Miss. Alice Elizabeth	3
374	Palsson, Miss. Stina Viola	3
386	Goodwin, Master. Sidney Leonard	5

Step 14: List the Passengers who did not Survive and had no Family Aboard

```
#EXP_NO_2
#Sanjay_S_241801246
no_family = df[(df["Survived"] == 0) &
               (df["SibSp"] == 0) &
               (df["Parch"] == 0)]
no_family[["Name", "Survived", "SibSp", "Parch"]]
```

	Name	Survived	SibSp	Parch
4	Allen, Mr. William Henry	0	0	0
5	Moran, Mr. James	0	0	0
6	McCarthy, Mr. Timothy J	0	0	0
12	Saundercock, Mr. William Henry	0	0	0
14	Vestrom, Miss. Hulda Amanda Adolfina	0	0	0
...	...	...	...	...
882	Dahlberg, Miss. Gerda Ulrika	0	0	0
883	Banfield, Mr. Frederick James	0	0	0
884	Sutehall, Mr. Henry Jr	0	0	0
886	Montvila, Rev. Juozas	0	0	0
890	Dooley, Mr. Patrick	0	0	0

374 rows × 4 columns

Step 15: Find the Top 5 Oldest Passengers who Survived

```
#EXP_NO_2
#Sanjay_S_241801246
top5 = df[df["Survived"] == 1].sort_values(by="Age", ascending=False)
top5[["Name", "Age"]].head(5)
```

	Name	Age
630	Barkworth, Mr. Algernon Henry Wilson	80.0
275	Andrews, Miss. Kornelia Theodosia	63.0
483	Turkula, Mrs. (Hedwig)	63.0
570	Harris, Mr. George	62.0
829	Stone, Mrs. George Nelson (Martha Evelyn)	62.0

Step 16: List Passengers who Paid Zero Fare

```
#EXP_NO_2
#Sanjay_S_241801246
zero_fare = df[df["Fare"] == 0]
zero_fare[["Name", "Fare"]]
```

	Name	Fare
179	Leonard, Mr. Lionel	0.0
263	Harrison, Mr. William	0.0
271	Tornquist, Mr. William Henry	0.0
277	Parkes, Mr. Francis "Frank"	0.0
302	Johnson, Mr. William Cahoone Jr	0.0
413	Cunningham, Mr. Alfred Fleming	0.0
466	Campbell, Mr. William	0.0
481	Frost, Mr. Anthony Wood "Archie"	0.0
597	Johnson, Mr. Alfred	0.0
633	Parr, Mr. William Henry Marsh	0.0

Step 17: Split the Dataset for Training and Testing

```
#EXP_NO_2
#Sanjay_S_241801246
X = df.drop("Survived", axis=1)
y = df["Survived"]

X = pd.get_dummies(X)

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

print("Training Data Shape:", X_train.shape)
print("Testing Data Shape:", X_test.shape)
```

```
Training Data Shape: (712, 1731)
Testing Data Shape: (179, 1731)
```

The Titanic dataset was successfully loaded and analyzed. Dataset information and summary statistics were displayed, missing values in Age, Cabin, and Embarked were handled, and passenger-class distribution was visualized. Passenger records were filtered using the specified conditions. Finally, the dataset was split into 712 training records and 179 testing records.

EXPERIMENT 3: MODEL PLANNING AND BUILDING

AIM: To implement model planning and building on the Advertising dataset by exploring and preprocessing data, training Linear Regression, Polynomial Regression, and Support Vector Regression models, evaluating their predictive performance using metrics (MAE, MSE, RMSE), and segmenting the data using K-Means Clustering with visual plots.

ALGORITHM:

1. Load the Advertising dataset from Kaggle into a DataFrame.
2. Explore and preprocess the data by checking for null values and data types, selecting features (TV, Radio, Newspaper) and the target variable (Sales), and plotting a pair plot on the features.
3. Split the dataset into training and testing sets using train_test_split().
4. Train a Linear Regression model on the training data and make predictions on the test set.
5. Train a Polynomial Regression model by constructing a pipeline with PolynomialFeatures(degree=2) and LinearRegression, fitting it on the training set, and predicting on the test set.
6. Train a Support Vector Regression (SVR) model using an RBF kernel on the training set and compute predictions on the test set.
7. Evaluate the models by computing MAE, MSE, and RMSE using mean_absolute_error, mean_squared_error, and np.sqrt(), and visually compare all models on a single bar graph.
8. Apply K-Means Clustering by fitting KMeans(n_clusters=3) on the feature set and assigning the resulting cluster labels to the dataset.
9. Visualize the clusters using seaborn.scatterplot() (e.g., TV vs Sales colored by cluster label).

Step 1: Load the Advertising Dataset

```
#EXP_NO_3
#SANJAY_S_241801246
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVR
from sklearn.metrics import mean_absolute_error, mean_squared_error
from sklearn.cluster import KMeans


df = pd.read_csv("advertising.csv")
df.head(5)
```

	TV	Radio	Newspaper	Sales
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	12.0
3	151.5	41.3	58.5	16.5
4	180.8	10.8	58.4	17.9

Step 2: Explore and Preprocess Data

```
#EXP_NO_3
#SANJAY_S_241801246
print(df.shape)
```

```
(200, 4)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype  
---  ------     --------------  -----  
 0   TV         200 non-null    float64
 1   Radio      200 non-null    float64
 2   Newspaper  200 non-null    float64
 3   Sales      200 non-null    float64
dtypes: float64(4)
memory usage: 6.4 KB
```

```
print(df.isnull().sum())
```

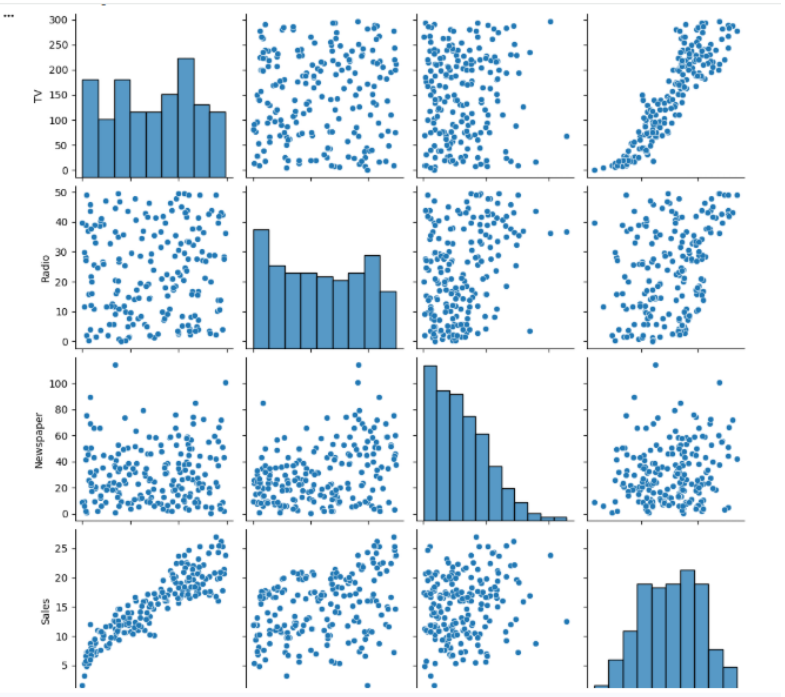
```
TV           0
Radio        0
Newspaper    0
Sales        0
dtype: int64
```

```
dtype: int64
```

```
print(df.describe())
```

```
               TV       Radio   Newspaper       Sales
count  200.000000  200.000000  200.000000  200.000000
mean   147.042500   23.264000   30.554000   15.130500
std     85.854236   14.846809   21.778621    5.283892
min      0.700000    0.000000    0.300000    1.600000
25%     74.375000    9.975000   12.750000   11.000000
50%    149.750000   22.900000   25.750000   16.000000
75%    218.825000   36.525000   45.100000   19.050000
max    296.400000   49.600000  114.000000   27.000000
```

```
sns.pairplot(df)
```



Step 3: Split the Dataset

```
#EXP_NO_3
#SANJAY_S_241801246
X = df[['TV', 'Radio', 'Newspaper']]
y = df['Sales']
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)
```

```
Training Data: (160, 3)
Testing Data: (40, 3)
```

Step 4: Train Linear Regression

```
#EXP_NO_3
#SANJAY_S_241801246
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)
linear_pred = linear_model.predict(X_test)
print("Linear Regression Predictions:")
print(linear_pred[:5])
```

```
Linear Regression Predictions:
[17.0347724  20.40974033 23.72398873  9.27278518 21.68271879]
```

Step 5: Train Polynomial Regression

```
#EXP_NO_3
#SANJAY_S_241801246
poly_model = make_pipeline(
    PolynomialFeatures(degree=2),
    LinearRegression()
)
poly_model.fit(X_train, y_train)
poly_pred = poly_model.predict(X_test)
print("Polynomial Regression Predictions:")
print(poly_pred[:5])
```

```
Polynomial Regression Predictions:
[17.562438   21.96285664 22.07580481  7.34916293 23.23684435]
```

Step 6: Train Support Vector Regression (SVR)

```
#EXP_NO_3
#SANJAY_S_241801246
svr_model = SVR(kernel='rbf')
svr_model.fit(X_train, y_train)
svr_pred = svr_model.predict(X_test)
print("SVR Predictions:")
print(svr_pred[:5])
```

```
SVR Predictions:
[17.15330196 19.49875585 20.96245814  8.60209215 19.88553201]
```

Step 7: Evaluate the Models

```
#EXP_NO_3
#SANJAY_S_241801246
models = {
    "Linear Regression": linear_pred,
    "Polynomial Regression": poly_pred,
    "SVR": svr_pred
}

mae_values = []
mse_values = []
rmse_values = []

for name, pred in models.items():
    mae = mean_absolute_error(y_test, pred)
    mse = mean_squared_error(y_test, pred)
    rmse = np.sqrt(mse)

    mae_values.append(mae)
    mse_values.append(mse)
    rmse_values.append(rmse)

    print(name)
    print("MAE :", mae)
    print("MSE :", mse)
    print("RMSE:", rmse)
    print()

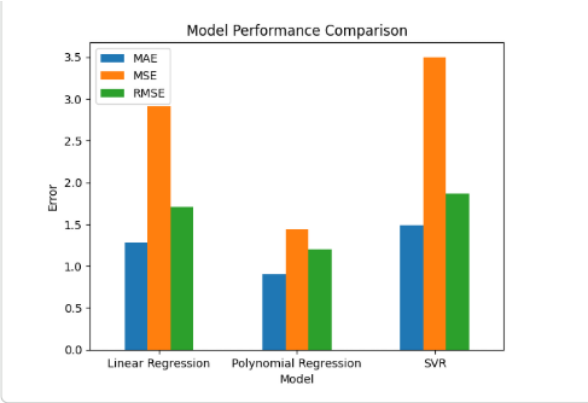
# Comparison bar graph
results = pd.DataFrame({
    "Model": list(models.keys()),
    "MAE": mae_values,
    "MSE": mse_values,
    "RMSE": rmse_values
})

results.set_index("Model").plot(kind="bar")
plt.title("Model Performance Comparison")
plt.ylabel("Error")
plt.xticks(rotation=0)
plt.show()
```

```
Linear Regression
MAE : 1.2748262109549338
MSE : 2.907756910271089 6
RMSE: 1.7052146229349223

Polynomial Regression
MAE : 0.9033713382792442
MSE : 1.44254263690159
RMSE: 1.2010589647896517

SVR
MAE : 1.4836223881587627
MSE : 3.499087197255254
RMSE: 1.8705847206836834
```



Step 8: Apply K-Means Clustering

```
#EXP_NO_3
#SANJAY_S_241801246
kmeans = KMeans(n_clusters=3, random_state=42)
df['Cluster'] = kmeans.fit_predict(
    df[['TV', 'Radio', 'Newspaper']]
)
print(df[['TV', 'Radio', 'Newspaper', 'Cluster']].head())
```

```
      TV  Radio  Newspaper  Cluster
0  230.1   37.8       69.2        0
1   44.5   39.3       45.1        1
2   17.2   45.9       69.3        1
3  151.5   41.3       58.5        2
4  180.8   10.8       58.4        2
```

Step 9: Visualize the Clusters

```
#EXP_NO_3
#SANJAY_S_241801246
sns.scatterplot(
    x='TV',
    y='Sales',
    hue='Cluster',
    data=df
)
plt.title("K-Means Clusters: TV vs Sales")
plt.show()
```



RESULT: The model planning and building exercise was successfully implemented. Linear Regression, Polynomial Regression, and Support Vector Regression models were built and evaluated using MAE, MSE, and RMSE metrics, and data clustering was performed and visualized using K-Means.

EXPERIMENT 4 : INFORMATION RETRIEVAL USING NLTK

AIM: To design and implement an information retrieval system using NLTK, TF-IDF vectorization, and Cosine Similarity on the Amazon Fine Food Reviews dataset by preprocessing customer review text, converting text to numerical feature vectors, and retrieving the top 5 most relevant reviews for user search queries.

ALGORITHM:

1. Import required Python libraries (pandas, nltk, TfidfVectorizer from sklearn.feature_extraction.text, cosine_similarity from sklearn.metrics.pairwise) and download necessary NLTK packages (e.g., stopwords, punkt).
2. Load the Amazon Fine Food Reviews Dataset from Kaggle into a Pandas DataFrame.
3. Clean the dataset by selecting the Text column, removing null or missing values, dropping duplicate reviews, and retaining the first 10,000 reviews for faster processing.
4. Define a text preprocessing function to convert text to lowercase, strip punctuation, numbers, and special characters, tokenize text into words, remove English stopwords, and rejoin tokens into a cleaned text string.
5. Apply the preprocessing function to all customer reviews and save the processed text in a new column named Cleaned_Text.
6. Display original and cleaned versions of any five sample reviews.
7. Convert the Cleaned_Text column into TF-IDF feature vectors using TfidfVectorizer and display the resulting matrix shape and total vocabulary size.
8. Define an Information Retrieval function that:
    - Accepts an input search query.
    - Preprocesses the query using the text preprocessing pipeline.
    - Transforms the query into a TF-IDF vector.
    - Computes Cosine Similarity between the query vector and all document vectors.
    - Ranks and extracts the Top 5 most similar reviews displaying similarity scores, original review text, and cleaned review text.

9. Evaluate the retrieval system using sample test queries: "great product with fast shipping", "disappointed", "excellent taste", and "poor packaging".
10. Compare the retrieved results across queries to verify relevance.

Information Retrieval (IR) using NLTK

Step 1: Import Libraries and Download NLTK Resources

```
#Information_Retrival _2
#Sanjay S(241801246)
import nltk
import pandas as pd
import string
import re
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

```
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger.zip.
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger_eng.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
True
```

Step 2: Load the 20 Newsgroups Dataset

```
#Information_Retrival _2
#SANJAY_S_241801246
newsgroups_data = fetch_20newsgroups(
    subset='train',
    remove=('headers', 'footers', 'quotes')
)
documents = newsgroups_data.data[:1000]
df = pd.DataFrame({'text': documents})
print(df.head(2).to_markdown())
```

```
|    | text                                                
```

Step 3: Load and Extract Stop Words

```
#Information_Retrival _2
#SANJAY_S_241801246
stop_words = set(stopwords.words('english'))
print("Sample Stop Words:")
print(list(stop_words)[:5])
```

```
Sample Stop Words:
['them', "they'll", 'doesn', 'so', 'whom']
```

Step 4: Extract Stop Words from Text

```
#Information_Retrival _2
#SANJAY_S_241801246
def extract_stop_words(text):
    text = text.lower()
    tokens = word_tokenize(text)
    return [word for word in tokens if word in stop_words and word.isalpha()]
stop_words_list = df['text'].apply(extract_stop_words)
stop_words_list
```

	text
0	[i, was, if, out, there, me, on, this, i, the,...
1	[a, of, who, their, have, their, for, this, a,...
2	[my, up, the, this, after, as, a, in, i, in, t...
3	[do, you, have, i, to, some, about, this]
4	[from, by, a, my, is, that, the, are, in, the,...
...	...
995	[i, have, a, and, with, an]
996	[you, do, they, have, out, y, of, to, the, me,...

Step 5: Preprocess the Text

```
#Information_Retrival _2
#SANJAY_S_241801246
def preprocess(text):
    text = text.lower()
    text = re.sub(r'[^a-z\s]', '', text)
    tokens = word_tokenize(text)
    tokens = [w for w in tokens if w not in stop_words]
    return ' '.join(tokens)
df['clean_text'] = df['text'].apply(preprocess)
df['clean_text']
```

...

	clean_text
0	wondering anyone could enlighten car saw day d...
1	fair number brave souls upgraded si clock osci...
2	well folks mac plus finally gave ghost weekend...
3	weiteks addressphone number id like get inform...
4	article cowcbnpworldstdcom tombakerworldstdcom...
...	...
995	vt vt compatible terminal external hyess modem...
996	obviously really dont know spoken cf secy stat...
997	noticed exact phenomenon occurs lciii perhaps ...
998	essential tremor progressive hereditary tremor...
999	sorry posting email keeps bouncing maybe help ...

Step 6: TF-IDF Vectorization

```
#Information_Retrival _2
#SANJAY_S_241801246
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(df['clean_text'])
```

Step 7: Define the Search Query

```
#Information_Retrival _2
#SANJAY_S_241801246
query = "space shuttle launch"
query_clean = preprocess(query)
query_vec = vectorizer.transform([query_clean])
```

Step 8: Compute Cosine Similarity

```
#Information_Retrival _2
#SANJAY_S_241801246
similarities = cosine_similarity(
    query_vec,
    tfidf_matrix
).flatten()
print(similarities[:5])
```

```
[0.         0.         0.         0.         0.10156444]
```

Step 9: Rank the Top 5 Documents

Step 9: Rank the Top 5 Documents

```
#Information_Retrival _2
#SANJAY_S_241801246
top_indices = similarities.argsort()[-5:][::-1]
print("\nTop 5 Relevant Documents for Query:", query)
for i in top_indices:
    print(
        f"\nDoc {i+1} | Score: {similarities[i]:.4f}\n"
        f"{df.loc[i, 'text'][:400]}...\n"
    )
```

```
Doc 154 | Score: 0.3939
Archive-name: space/schedule
Last-modified: $Date: 93/04/01 14:39:23 $

SPACE SHUTTLE ANSWERS, LAUNCH SCHEDULES, TV COVERAGE

    SHUTTLE LAUNCHINGS AND LANDINGS; SCHEDULES AND HOW TO SEE THEM

    Shuttle operations are discussed in the Usenet group sci.space.shuttle,
    and Ken Hollis (gandalf@pro-electric.cts.com) posts a compressed version
    of the shuttle manifest (launch dates and other i...


Doc 344 | Score: 0.2446

In fact, you probably want to avoid US Government anything for such a
project.  The pricetag is invariably too high, either in money or in
hassles.

The important thing to realize here is that the big cost of getting to
the Moon is getting into low Earth orbit.  Everything else is practically
down in the noise.  The only part of getting to the Moon that poses any
new problems, beyond what you fac...


Doc 60 | Score: 0.2308
Archive-name: space/new_probes
Last-modified: $Date: 93/04/01 14:39:17 $

UPCOMING PLANETARY PROBES - MISSIONS AND SCHEDULES

    Information on upcoming or currently active missions not mentioned below
    would be welcome. Sources: NASA fact sheets, Cassini Mission Design
    team, ISAS/NASDA launch schedules, press kits.


    ASUKA (ASTRO-D) - ISAS (Japan) X-ray astronomy satellite, launched i...


Doc 357 | Score: 0.2017
According to a Software engineering professor here, what was actually rated
level five was an ibm unit which produced part of the software for the shuttle,
by not means all of it.

Interesting note: 90% of the software development groups surveyed were at
level 1. The ibm shuttle groups was the *only* one at level 5!
...


Doc 233 | Score: 0.1982


SPECIFIC:
```

Step 10: POS Tagging

```
#Information_Retrival _2
#SANJAY_S_241801246
for i in range(3):
    tokens = word_tokenize(df.loc[i, 'clean_text'])
    tagged = nltk.pos_tag(tokens)
    print(f"\nDocument {i+1} POS Tags:")
    print(tagged[:10])
```

```
Document 1 POS Tags:
[('wondering', 'VBG'), ('anyone', 'NN'), ('could', 'MD'), ('enlighten', 'VB'), ('car', 'NN'), ('saw', 'JJ'), ('day', 'NN'), ('door', 'VB'), ('sports', 'NNS'), ('car', 'NN')]

Document 2 POS Tags:
[('fair', 'JJ'), ('number', 'NN'), ('brave', 'VBP'), ('souls', 'NNS'), ('upgraded', 'VBD'), ('si', 'JJ'), ('clock', 'NN'), ('oscillator', 'NN'), ('shared', 'VBD'), ('experiences', 'NNS')]

Document 3 POS Tags:
[('well', 'RB'), ('folks', 'NNS'), ('mac', 'VBP'), ('plus', 'CC'), ('finally', 'RB'), ('gave', 'VBD'), ('ghost', 'VBN'), ('weekend', 'NN'), ('starting', 'VBG'), ('life', 'NN')]
```

RESULT: An Information Retrieval system using NLTK, TF-IDF, and Cosine Similarity was successfully built and evaluated. Customer reviews were cleaned, preprocessed, vectorized, and the system effectively retrieved and ranked the top 5 relevant reviews with similarity scores for various search queries.

EXPERIMENT 5 : INFORMATION RETRIEVAL USING SpaCy

AIM:

To implement an information retrieval system using spaCy and TF-IDF by preprocessing text, removing stop words, analyzing part-of-speech tags, and retrieving the most relevant documents for a user query using cosine similarity.

ALGORITHM:

1. Import spaCy, Pandas, fetch_20newsgroups, TfidfVectorizer, and cosine_similarity.

2. Load the 20 Newsgroups dataset and select the first 500 documents, excluding headers, footers, and quoted text.

3. Define a preprocessing function to convert text to lowercase, tokenize it with spaCy, remove stop words and non-alphabetic tokens, and use lemmas.

4. Store the original text and preprocessed text in a Pandas DataFrame.

5. Extract stop words from the documents using spaCy and display sample results.

6. Print part-of-speech tags for tokens in the first three documents.

7. Convert the cleaned documents into TF-IDF vectors using TfidfVectorizer.

8. Enter the query “NASA space launch” and preprocess it using the same text-preprocessing function.

9. Transform the query into a TF-IDF vector and calculate cosine similarity between the query and all documents.

10. Sort the similarity scores and display the top five most relevant documents.

Information Retrieval (IR) using spaCy

Step 1: Import Libraries and Load spaCy Model

```
#Information_Retrieval_3
#Shailesh kumar V(241801256)
import spacy
import pandas as pd
import string
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
nlp = spacy.load("en_core_web_sm")
```

Step 2: Load Dataset

```
#Information_Retrieval_3
#SANJAY_S_241801246
newsgroups = fetch_20newsgroups(
    subset='all',
    remove=('headers', 'footers', 'quotes')
)
documents = newsgroups.data[:500]
```

Step 3: Preprocessing with spaCy

```
#Information_Retrieval_3
#SANJAY_S_241801246
def preprocess_spacy(text):
    doc = nlp(text.lower())
    tokens = [
        token.lemma_
        for token in doc
        if token.is_alpha and not token.is_stop
    ]
    return ' '.join(tokens)
```

Step 4: Apply Preprocessing

```
#Information_Retrieval_3
#SANJAY_S_241801246
df = pd.DataFrame({'Text': documents})
df['clean_text'] = df['Text'].apply(preprocess_spacy)
df['clean_text'].head()
```

	clean_text
0	sure basher pen fan pretty confused lack kind ...
1	brother market high performance video card sup...
2	finally say dream mediterranean new area great...
3	think scsi card dma transfer disk scsi card dm...
4	old jasmine drive use new system understanding...

dtype: object

Step 5: Extract Stop Words

```
#Information_Retrieval_3
#SANJAY_S_241801246
def extract_stop_words(text):
    doc = nlp(text.lower())
    stop_words_in_text = [
        token.lemma_
        for token in doc
        if token.is_stop and token.is_alpha
    ]
    return ' '.join(stop_words_in_text)
df['stop_words'] = df['Text'].apply(extract_stop_words)
df['stop_words'].head()
```

	stop_words
0	I be some of be about the of any of about the ...
1	my be in the for a that with do anyone have on...
2	you what you about that be the will be after s...
3	it the do the not the the can do from any of t...
4	I have an which I can not with my my be that I...

dtype: object

Step 6: POS Tagging

```
#Information_Retrieval_3
#SANJAY_S_241801246
for i, doc in enumerate(df['Text'][:3]):
    print(f"\nDocument {i+1} POS Tags:")
    spacy_doc = nlp(doc)
    print([(token.text, token.pos_) for token in spacy_doc[:10]])
```

```
Document 1 POS Tags:
[('\n\n', 'SPACE'), ('I', 'PRON'), ('am', 'AUX'), ('sure', 'ADJ'), ('some', 'DET'), ('bashers', 'NOUN'), ('of', 'ADP'), ('Pens', 'PROPN'), ('fans', 'NOUN'), ('are', 'AUX')]

Document 2 POS Tags:
[('My', 'PRON'), ('brother', 'NOUN'), ('is', 'AUX'), ('in', 'ADP'), ('the', 'DET'), ('market', 'NOUN'), ('for', 'ADP'), ('a', 'DET'), ('high', 'ADJ'), ('-', 'PUNCT')]

Document 3 POS Tags:
[('\n\n\n\n\t', 'SPACE'), ('Finally', 'ADV'), ('you', 'PRON'), ('said', 'VERB'), ('what', 'PRON'), ('you', 'PRON'), ('dream', 'VERB'), ('about', 'ADP'), ('.', 'PUNCT'), ('Mediterranean', 'PROPN')]
```

Step 7: Vectorization using TF-IDF

+ Code + Text

```
#Information_Retrieval_3
#SANJAY_S_241801246
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(df['clean_text'])
```

Step 8: Define the Query

```
#Information_Retrieval_3
#SANJAY_S_241801246
query = "NASA space launch"
query_clean = preprocess_spacy(query)
query_vec = vectorizer.transform([query_clean])
```

Step 9: Calculate Similarity and Rank Documents

```
#Information_Retrieval_3
#SANJAY_S_241801246
similarities = cosine_similarity(
    query_vec,
    tfidf_matrix
).flatten()
top_indices = similarities.argsort()[-5:][::-1]
```

Step 10: Display Top 5 Results

```
#Information_Retrieval_3
#SANJAY_S_241801246
print("\nTop 5 Relevant Documents:")
for i in top_indices:
    print(df.loc[i, 'Text'][:300])
    print()
```

```
Top 5 Relevant Documents:
I am posting this for a friend without internet access. Please inquire
to the phone number and address listed.
---------------------------------------------------------------------

"Space: Teaching's Newest Frontier"
Sponsored by the Planetary Studies Foundation

The Planetary Studies Foundation is

: >Why not build a inflatable space dock.

: If you're doing large-scale satellite servicing, being able to do it in
: a pressurized hangar makes considerable sense.  The question is whether
: anyone is going to be doing large-scale satellite servicing in the near
: future, to the point of justifyin

"Space Station Redesign Leader Says Cost Goal May Be
Impossible"

Today (4/6) the Washington Post ran an article with the
headline shown above. The article starts with "A leader
of the NASA team in charge of redesigning the planned
space station said yesterday the job is tough and may
be impossible.




Phil> Didn't one of the early jet fighters have these?  I also think
Phil> the germans did some work on these in WWII.

The NACA came up with them before World War II.  NASA is directly
descended from the NACA, with space added in.

You'll notice that I didn't mention sweep wings even though t

As for SF and advertising in space. There is a romantic episode
in Mead's "The Big Ball of Wax" where the lovers are watching
the constellation Pepsi Cola rising over the horizon and noting
the some 'stars' had slipped cause the Teamsters were on strike.

This was the inspiration for my article on
```

RESULT:

The information retrieval system using spaCy and TF-IDF was executed successfully. Text was preprocessed using tokenization, stop-word removal, and lemmatization; part-of-speech tags were displayed for sample documents. The query “NASA space launch” was compared with the document collection using cosine similarity, and the top five relevant documents were displayed

EXPERIMENT -6a : Exploratory Data Analysis

AIM: To perform Exploratory Data Analysis (EDA) on the Netflix Titles dataset using Python and data visualization tools to analyze content distribution, ratings, release and addition trends, durations, regional contributions, textual descriptions, and genre co-occurrences.

ALGORITHM:

1. **Import required Python libraries including pandas, numpy, matplotlib.pyplot, seaborn, and wordcloud.**
2. **Download the netflix_titles dataset from Kaggle and load it into a Pandas DataFrame.**
3. **Explore the dataset structure by checking data types, summary statistics, missing values, and column info.**
4. **Perform Content Distribution Analysis:**
    - **Compare the counts of Movies vs. TV Shows using visualization plots.**
    - **Plot the distribution of ratings (e.g., TV-MA, PG, R) and identify countries with the highest content volume.**
5. **Conduct Time-Based Trend Analysis:**
    - **Plot release years and addition years to analyze Netflix's content growth trends over time.**
    - **Identify which months of the year Netflix adds the most content.**
6. **Conduct Duration & Content Type Analysis:**
    - **Plot the distribution of movie durations in minutes.**
    - **Plot the distribution of TV shows categorized by number of seasons.**
7. **Execute Genre & Categories Analysis:**
    - **Extract and plot the top 10 most common genres, top directors, and most frequent actors.**
    - **Analyze country-wise production of movies versus TV shows.**
    - **Determine the most common ratings specifically for Movies versus TV Shows.**
8. **Execute Text & Descriptions Analysis:**

- **Generate a word cloud on the description column to identify the most common words.**

9. **Determine the oldest and newest titles available on the platform.**
10. **Construct a genre correlation heatmap to analyze which genres frequently co-occur together.**

1. Download Netflix Titles Dataset and Convert It to DataFrame

```
#Exp_no_6a
#Sanjay_S_241801246
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from wordcloud import WordCloud
df = pd.read_csv("netflix_titles.csv")
df.head()
```

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	September 25, 2021	2020	PG-13	90 min	Documentaries	As her father nears the end of his life, filmm...
1	s2	TV Show	Blood & Water	NaN	Ama Qamata, Khosi Ngema, Gail Mabalane, Thaban...	South Africa	September 24, 2021	2021	TV-MA	2 Seasons	International TV Shows, TV Dramas, TV Mysteries	After crossing paths at a party, a Cape Town t...
2	s3	TV Show	Ganglands	Julien Leclercq	Sami Bouajila, Tracy Gotoas, Samuel Jouy, Nabi...	NaN	September 24, 2021	2021	TV-MA	1 Season	Crime TV Shows, International TV Shows, TV Act...	To protect his family from a powerful drug lor...
3	s4	TV Show	Jailbirds New Orleans	NaN	NaN	NaN	September 24, 2021	2021	TV-MA	1 Season	Docuseries, Reality TV	Feuds, flirtations and toilet talk go down amo...
4	s5	TV Show	Kota Factory	NaN	Mayur More, Jitendra Kumar, Ranjan Raj, Alam K...	India	September 24, 2021	2021	TV-MA	2 Seasons	International TV Shows, Romantic TV Shows, TV ...	In a city of coaching centers known to train I...

2. Explore the Dataset

```
#Exp_no_6a
#Sanjay_S_241801246
print("Dataset Shape:", df.shape)
```

```
Dataset Shape: (8807, 12)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8807 entries, 0 to 8806
Data columns (total 12 columns):
 #   Column        Non-Null Count  Dtype 
---  ------        --------------  ----- 
 0   show_id       8807 non-null   object
 1   type          8807 non-null   object
 2   title         8807 non-null   object
 3   director      6173 non-null   object
 4   cast          7982 non-null   object
 5   country       7976 non-null   object
 6   date_added    8797 non-null   object
 7   release_year  8807 non-null   int64 
 8   rating        8803 non-null   object
 9   duration      8804 non-null   object
 10  listed_in     8807 non-null   object
 11  description   8807 non-null   object
dtypes: int64(1), object(11)
memory usage: 825.8+ KB
```

```
print(df.isnull().sum())
```

```
show_id            0
type               0
title              0
director        2634
cast             825
country          831
date_added        10
release_year       0
rating             4
duration           3
listed_in          0
description        0
dtype: int64
```

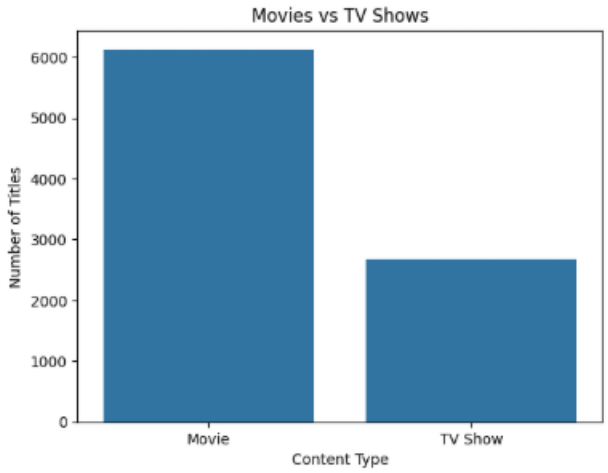
```
for col in ["director", "cast", "country", "rating", "date_added", "duration"]:
    df[col] = df[col].fillna("Unknown")

df["date_added"] = pd.to_datetime(df["date_added"], errors="coerce")
```

3a) How many Movies vs TV Shows are in the dataset?

```
#Exp_no_6a
#Sanjay_S_241801246
print(df["type"].value_counts())
sns.countplot(data=df, x="type")
plt.title("Movies vs TV Shows")
plt.xlabel("Content Type")
plt.ylabel("Number of Titles")
plt.show()
```

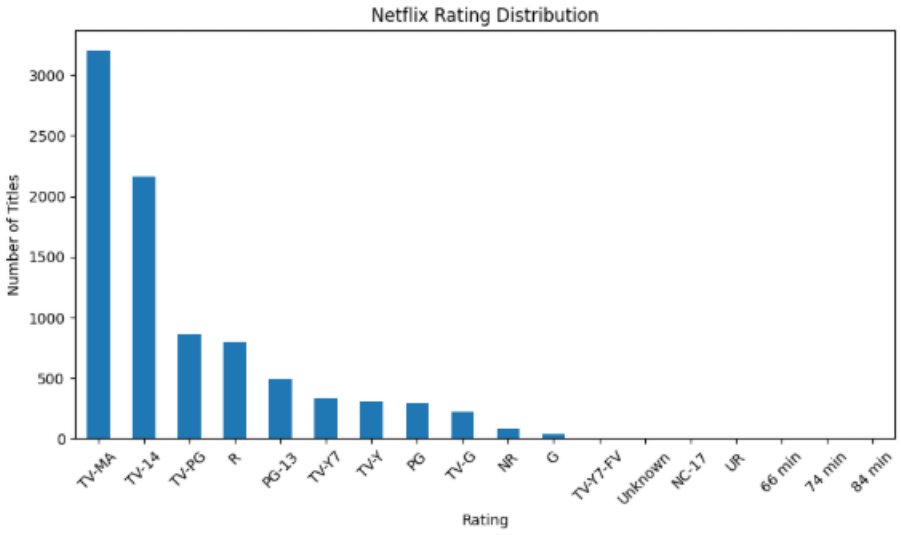
```
type
Movie      6131
TV Show    2676
Name: count, dtype: int64
```



3b) What is the distribution of ratings?

```
#Exp_no_6a
#Sanjay_S_241801246
rating_counts = df["rating"].value_counts()
print(rating_counts)
rating_counts.plot(kind="bar", figsize=(10, 5))
plt.title("Netflix Rating Distribution")
plt.xlabel("Rating")
plt.ylabel("Number of Titles")
plt.xticks(rotation=45)
plt.show()
```

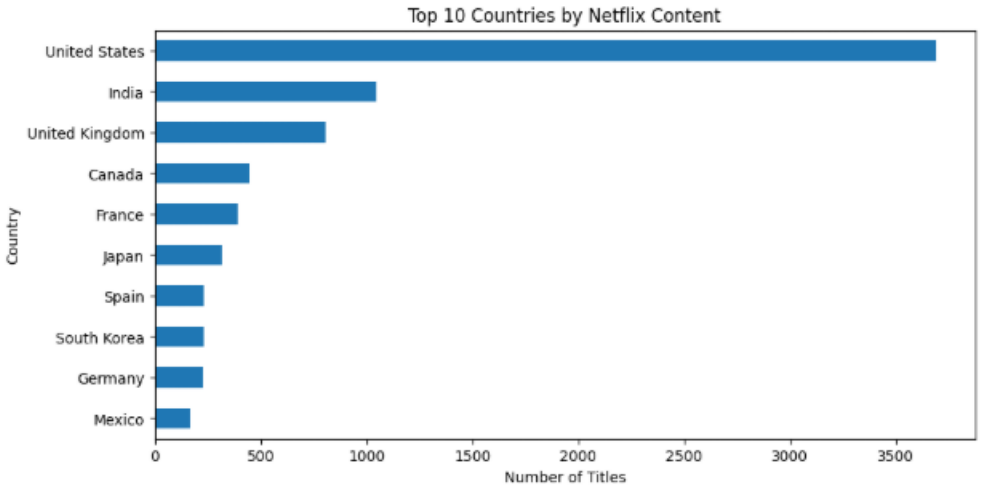
```
rating
TV-MA       3207
TV-14       2160
TV-PG        863
R            799
PG-13        490
TV-Y7        334
TV-Y         307
PG           287
TV-G         220
NR            80
G             41
TV-Y7-FV       6
Unknown        4
NC-17          3
UR             3
66 min         1
74 min         1
84 min         1
Name: count, dtype: int64
```



3c) Which countries have the most Netflix content?

```
#Exp_no_6a
#Sanjay_S_241801246
countries = df[df["country"] != "Unknown"]["country"].str.split(", ").explode()
top_countries = countries.value_counts().head(10)
print(top_countries)
top_countries.sort_values().plot(kind="barh", figsize=(10, 5))
plt.title("Top 10 Countries by Netflix Content")
plt.xlabel("Number of Titles")
plt.ylabel("Country")
plt.show()
```

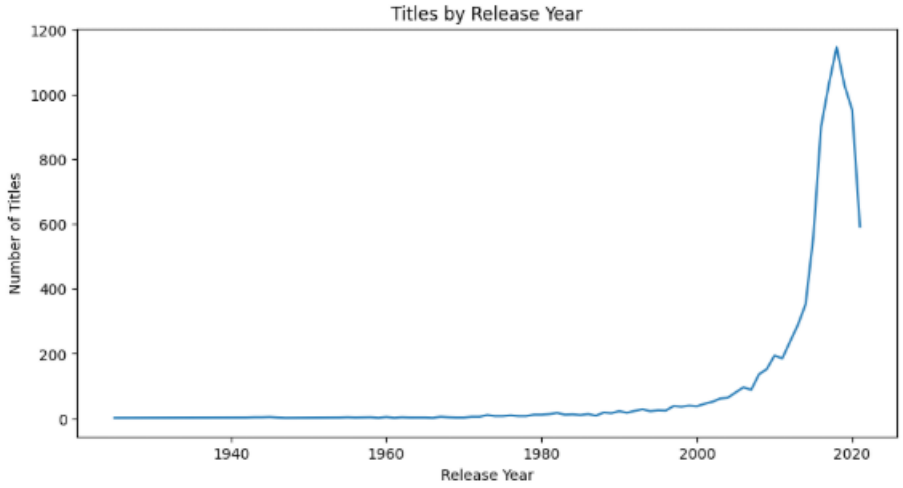
```
country
United States     3689
India             1046
United Kingdom     804
Canada             445
France             393
Japan              318
Spain              232
South Korea        231
Germany            226
Mexico             169
Name: count, dtype: int64
```



4a) In which years were the most shows/movies released?

```
#Exp_no_6a
#Sanjay_S_241801246
release_counts = df["release_year"].value_counts()
print("Top 10 Release Years:")
print(release_counts.head(10))
release_counts.sort_index().plot(figsize=(10, 5))
plt.title("Titles by Release Year")
plt.xlabel("Release Year")
plt.ylabel("Number of Titles")
plt.show()
```

```
Top 10 Release Years:
release_year
2018    1147
2017    1032
2019    1030
2020     953
2016     902
2021     592
2015     560
2014     352
2013     288
2012     237
Name: count, dtype: int64
```

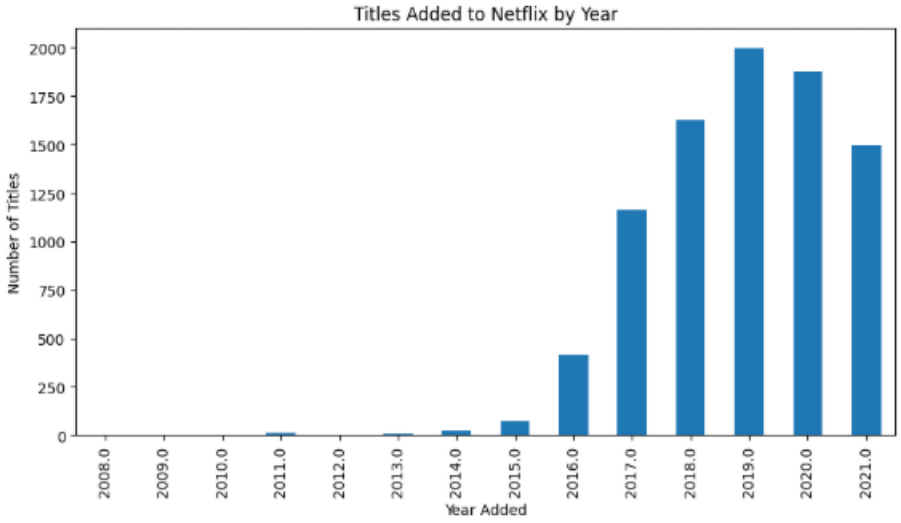


4b) In which years were titles added to Netflix?

4b) In which years were titles added to Netflix?

```
#Exp_no_6a
#Sanjay_S_241801246
df["added_year"] = df["date_added"].dt.year
added_counts = df["added_year"].value_counts().sort_index()
print(added_counts)
added_counts.plot(kind="bar", figsize=(10, 5))
plt.title("Titles Added to Netflix by Year")
plt.xlabel("Year Added")
plt.ylabel("Number of Titles")
plt.show()
```

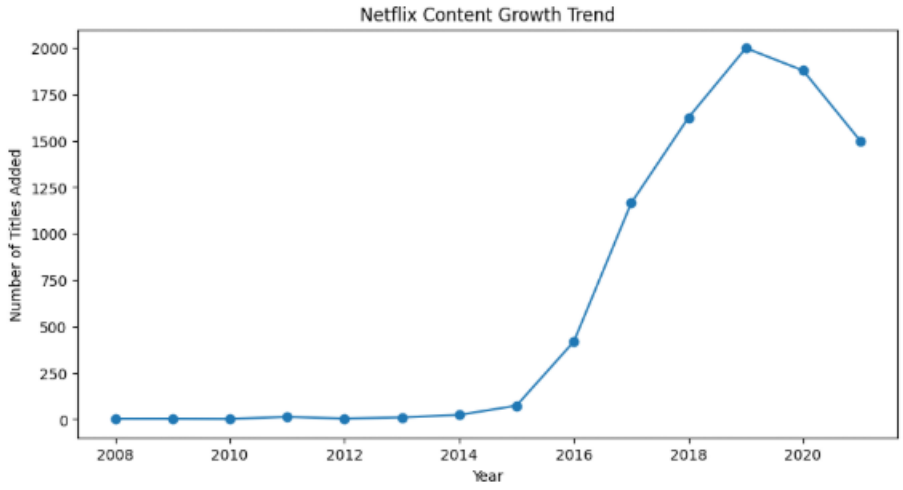
```
added_year
2008.0       2
2009.0       2
2010.0       1
2011.0      13
2012.0       3
2013.0      10
2014.0      23
2015.0      73
2016.0     418
2017.0    1164
2018.0    1625
2019.0    1999
2020.0    1878
2021.0    1498
Name: count, dtype: int64
```



4c) How has Netflix's content growth trend changed over time?

```
#Exp_no_6a
#Sanjay_S_241801246
growth = df.groupby("added_year").size().sort_index()
print(growth)
growth.plot(kind="line", marker="o", figsize=(10, 5))
plt.title("Netflix Content Growth Trend")
plt.xlabel("Year")
plt.ylabel("Number of Titles Added")
plt.show()
```

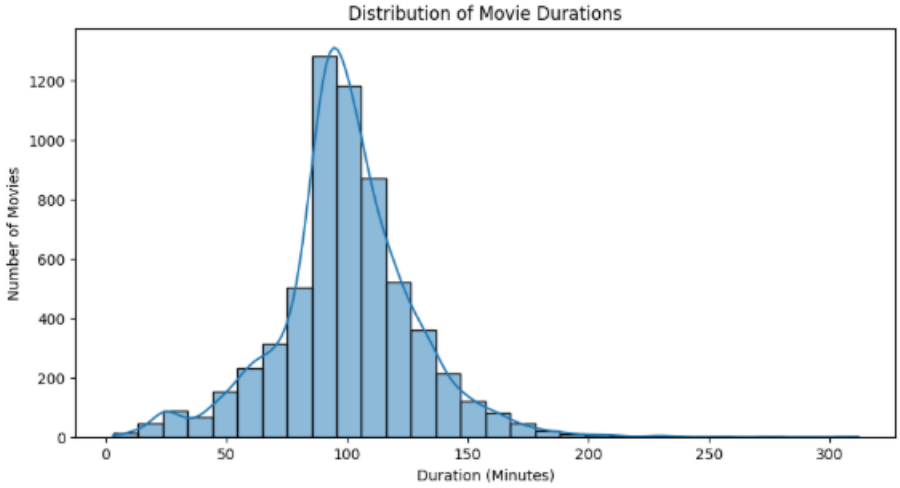
```
added_year
2008.0       2
2009.0       2
2010.0       1
2011.0      13
2012.0       3
2013.0      10
2014.0      23
2015.0      73
2016.0     418
2017.0    1164
2018.0    1625
2019.0    1999
2020.0    1878
2021.0    1498
dtype: int64
```



5a) What is the distribution of movie durations in minutes?

```
#Exp_no_6a
#Sanjay_S_241801246
movies = df[df["type"] == "Movie"].copy()
movies["duration_minutes"] = pd.to_numeric(
    movies["duration"].str.extract(r"(\d+)")[0],
    errors="coerce"
)
print(movies["duration_minutes"].describe())
plt.figure(figsize=(10, 5))
sns.histplot(movies["duration_minutes"].dropna(), bins=30, kde=True)
plt.title("Distribution of Movie Durations")
plt.xlabel("Duration (Minutes)")
plt.ylabel("Number of Movies")
plt.show()
```

```
count    6128.000000
mean       99.577187
std        28.290593
min         3.000000
25%        87.000000
50%        98.000000
75%       114.000000
max       312.000000
Name: duration_minutes, dtype: float64
```

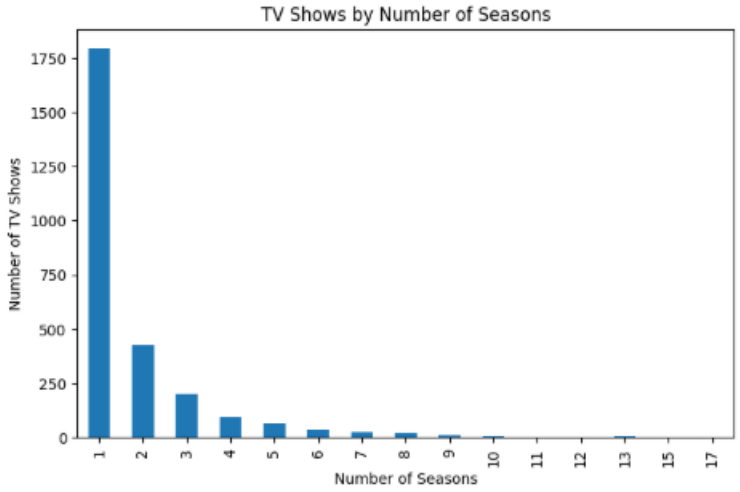


Duration (Minutes)

5b) How many TV shows exist per number of seasons?

```
#Exp_no_6a
#Sanjay_S_241801246
tv = df[df["type"] == "TV Show"].copy()
tv["seasons"] = pd.to_numeric(
    tv["duration"].str.extract(r"(\d+)")[0],
    errors="coerce"
)
season_counts = tv["seasons"].value_counts().sort_index()
print(season_counts)
season_counts.plot(kind="bar", figsize=(8, 5))
plt.title("TV Shows by Number of Seasons")
plt.xlabel("Number of Seasons")
plt.ylabel("Number of TV Shows")
plt.show()
```

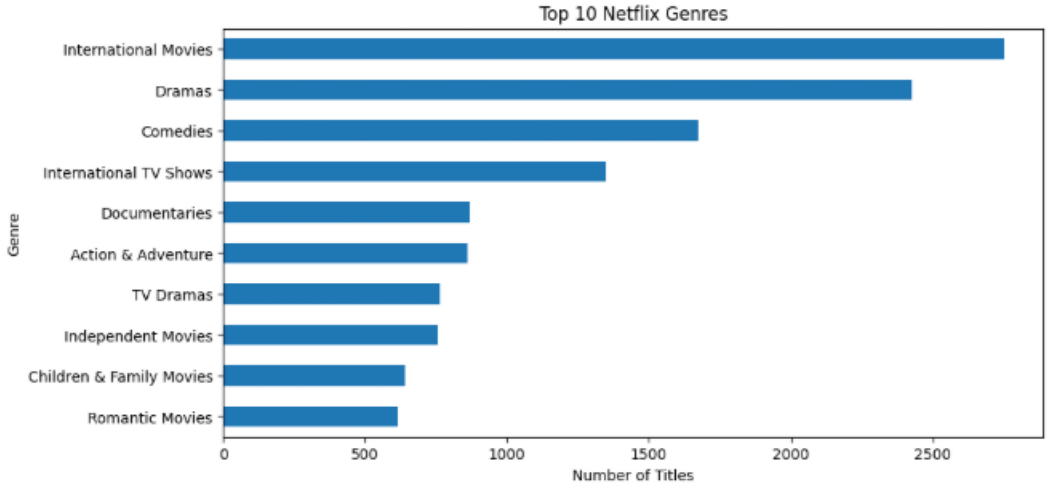
```
seasons
1     1793
2      425
3      199
4       95
5       65
6       33
7       23
8       17
9        9
10       7
11       2
12       2
13       3
15       2
17       1
Name: count, dtype: int64
```



6a) What are the top 10 most common genres on Netflix?

```
#Exp_no_6a
#Sanjay_S_241801246
genres = df["listed_in"].dropna().str.split(", ").explode()
top_genres = genres.value_counts().head(10)
print(top_genres)
top_genres.sort_values().plot(kind="barh", figsize=(10, 5))
plt.title("Top 10 Netflix Genres")
plt.xlabel("Number of Titles")
plt.ylabel("Genre")
plt.show()
```

```
listed_in
International Movies        2752
Dramas                      2427
Comedies                    1674
International TV Shows      1351
Documentaries                869
Action & Adventure           859
TV Dramas                    763
Independent Movies           756
Children & Family Movies     641
Romantic Movies              616
Name: count, dtype: int64
```

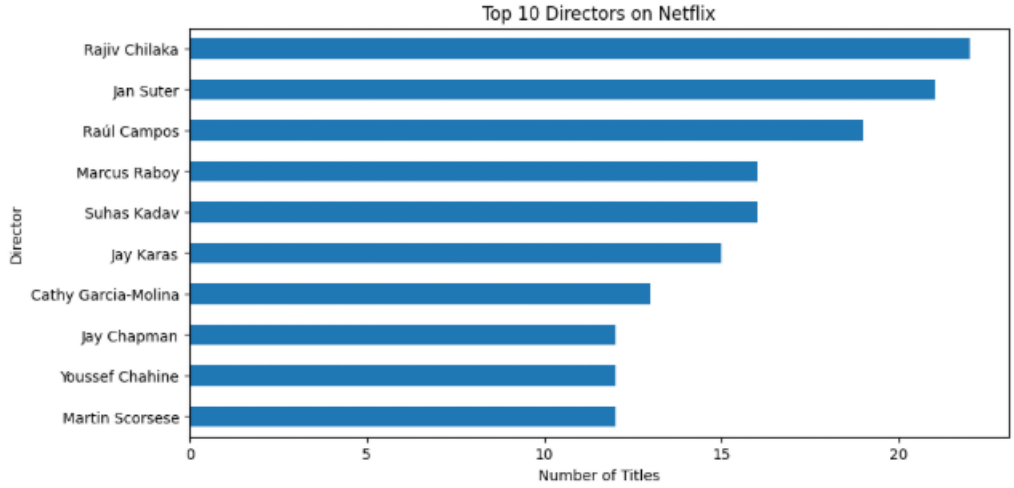


6b) Which directors have the most titles on Netflix?

6b) Which directors have the most titles on Netflix?

```
#Exp_no_6a
#Sanjay_S_241801246
directors = df[df["director"] != "Unknown"]["director"].str.split(", ").explode()
top_directors = directors.value_counts().head(10)
print(top_directors)
top_directors.sort_values().plot(kind="barh", figsize=(10, 5))
plt.title("Top 10 Directors on Netflix")
plt.xlabel("Number of Titles")
plt.ylabel("Director")
plt.show()
```

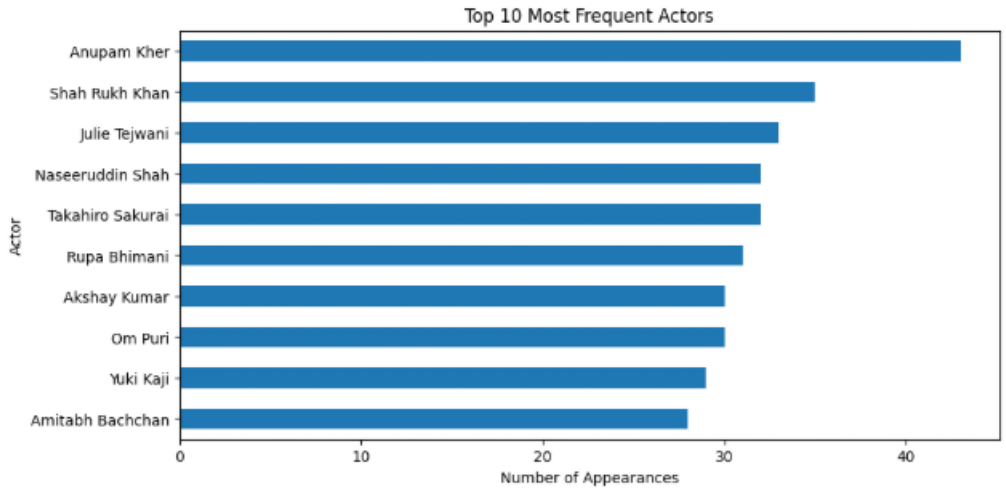
```
director
Rajiv Chilaka          22
Jan Suter              21
Raúl Campos            19
Suhas Kadav            16
Marcus Raboy           16
Jay Karas              15
Cathy Garcia-Molina    13
Martin Scorsese        12
Youssef Chahine        12
Jay Chapman            12
Name: count, dtype: int64
```



6c) Which actors appear most frequently across Netflix shows/movies?

```
#Exp_no_6a
#Sanjay_S_241801246
actors = df[df["cast"] != "Unknown"]["cast"].str.split(", ").explode()
top_actors = actors.value_counts().head(10)
print(top_actors)
top_actors.sort_values().plot(kind="barh", figsize=(10, 5))
plt.title("Top 10 Most Frequent Actors")
plt.xlabel("Number of Appearances")
plt.ylabel("Actor")
plt.show()
```

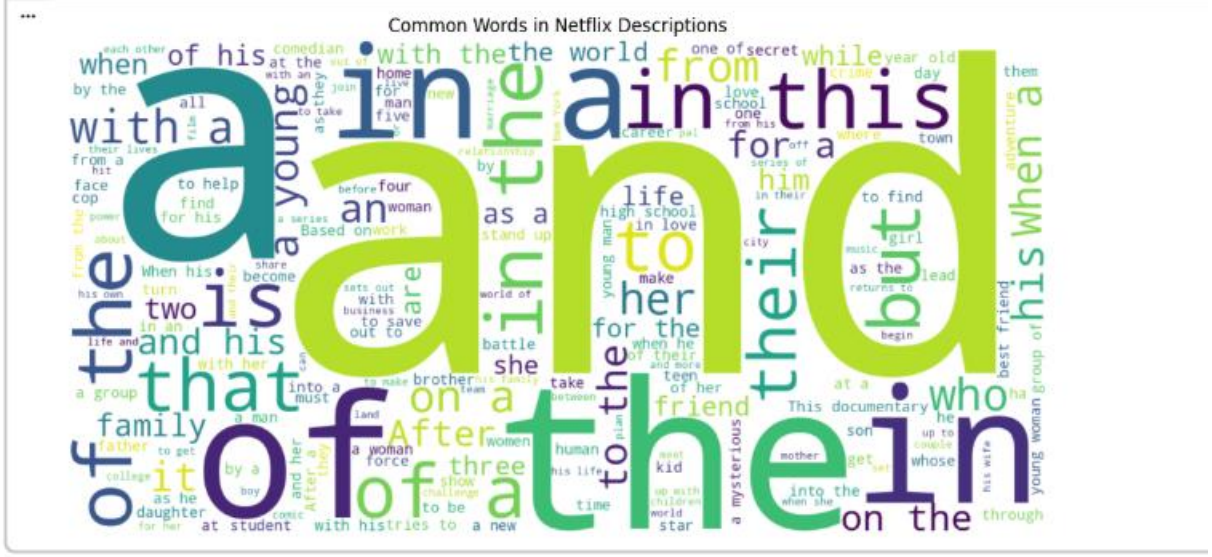
```
cast
Anupam Kher          43
Shah Rukh Khan       35
Julie Tejwani        33
Takahiro Sakurai     32
Naseeruddin Shah     32
Rupa Bhimani         31
Om Puri              30
Akshay Kumar         30
Yuki Kaji            29
Amitabh Bachchan     28
Name: count, dtype: int64
```



7. Text & Descriptions

7a) What are the most common words in the description field? (Word Cloud)

```
#Exp_no_6a
#Sanjay_S_241801246
text = " ".join(df["description"].dropna().astype(str))
wordcloud = WordCloud(
    width=1200,
    height=600,
    background_color="white",
    stopwords="english"
).generate(text)
plt.figure(figsize=(12, 6))
plt.imshow(wordcloud, interpolation="bilinear")
plt.axis("off")
plt.title("Common Words in Netflix Descriptions")
plt.show()
```

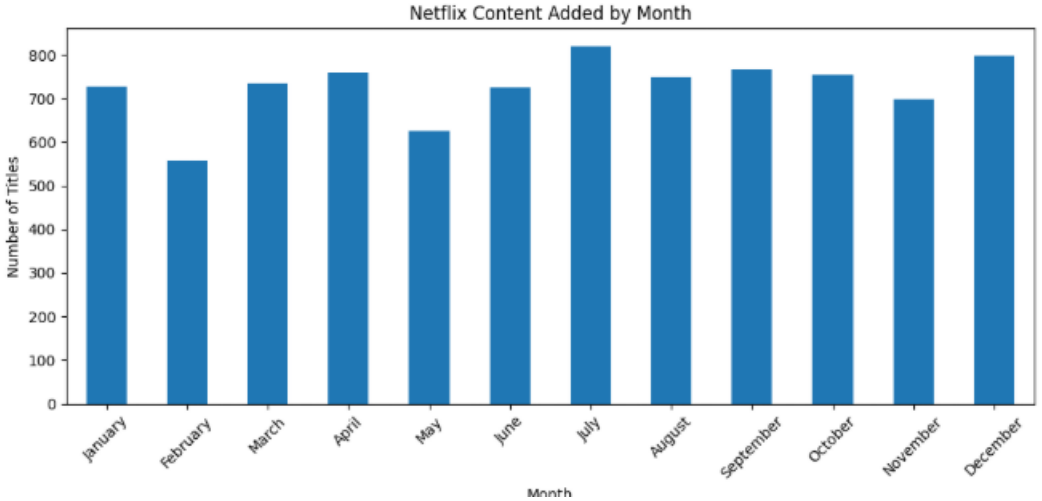


8. In Which Months of the Year Does Netflix Add the Most Content?

8. In Which Months of the Year Does Netflix Add the Most Content?

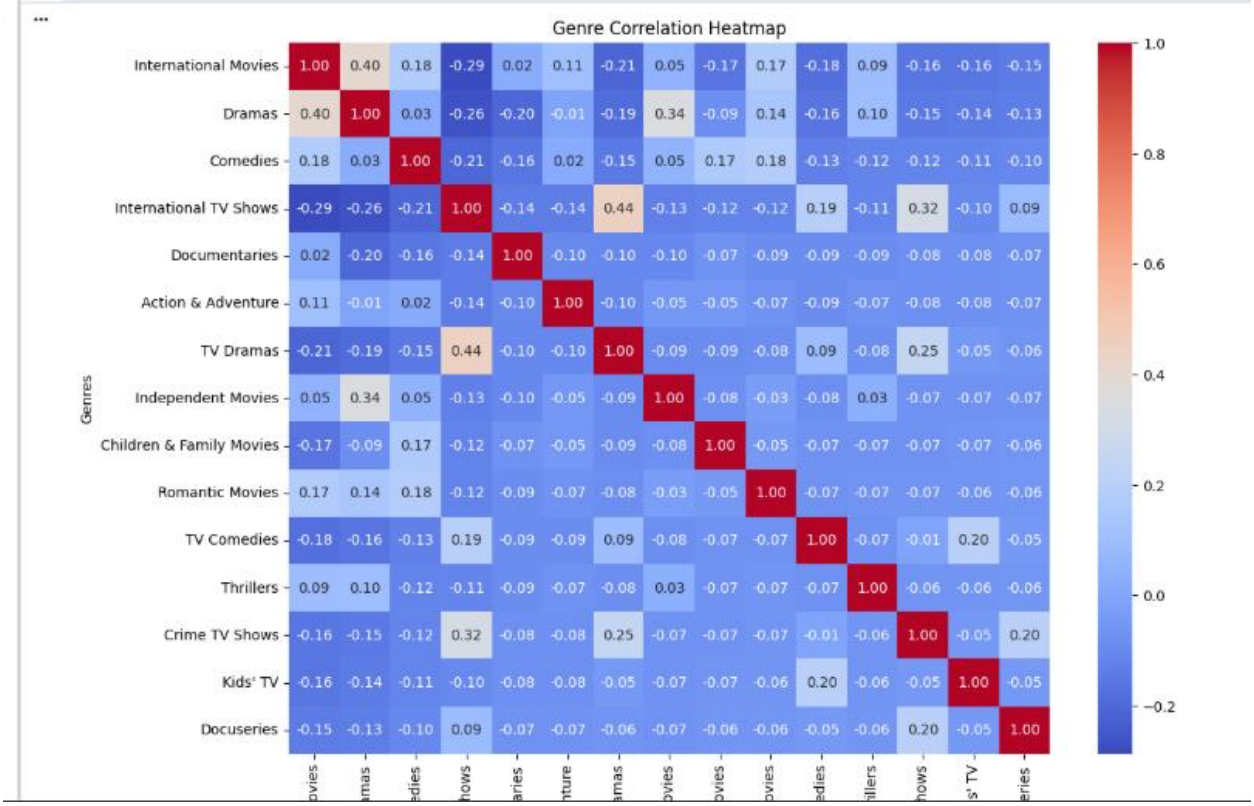
```
#Exp_no_6a
#Sanjay_S_241801246
import pandas as pd
import matplotlib.pyplot as plt
df["date_added"] = pd.to_datetime(df["date_added"], errors="coerce")
month_counts = df["date_added"].dt.month_name().value_counts()
month_order = [
    "January", "February", "March", "April",
    "May", "June", "July", "August",
    "September", "October", "November", "December"
]
month_counts = month_counts.reindex(month_order, fill_value=0)
print(month_counts)
month_counts.plot(kind="bar", figsize=(10, 5))
plt.title("Netflix Content Added by Month")
plt.xlabel("Month")
plt.ylabel("Number of Titles")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
date_added
January      727
February     557
March        734
April        759
May          626
June         724
July         819
August       749
September    765
October      755
November     697
December     797
Name: count, dtype: int64
```



12. Which Genres Frequently Co-occur Together? (Genre Correlation Heatmap)

```
#Exp_no_6a
#Sanjay_S_241801246
genre_data = df["listed_in"].fillna("").str.get_dummies(sep=", ")
top_genres = genre_data.sum().sort_values(ascending=False).head(15).index
genre_matrix = genre_data[top_genres]
correlation = genre_matrix.corr()
plt.figure(figsize=(12, 9))
sns.heatmap(
    correlation,
    annot=True,
    fmt=".2f",
    cmap="coolwarm"
)
plt.title("Genre Correlation Heatmap")
plt.xlabel("Genres")
plt.ylabel("Genres")
plt.show()
```



RESULT: Exploratory Data Analysis on the Netflix Titles dataset was successfully completed. Content characteristics, growth trends, duration distributions, key contributors, description keywords, and genre relationships were systematically extracted and visualized.

EXPERIMENT 6B : Time Series Data Analysis and Forecast

AIM: To analyze historical monthly airline passenger data, determine stationarity using the Augmented Dickey-Fuller (ADF) test, apply necessary transformations or differencing, build an ARIMA forecasting model, predict passenger demand for a test dataset, and evaluate forecast accuracy using MSE, MAE, and RMSE metrics.

ALGORITHM:

1. Import required Python libraries including pandas, numpy, matplotlib.pyplot, statsmodels.tsa.stattools (for adfuller), statsmodels.tsa.arima.model (for ARIMA), and sklearn.metrics.
2. Download the airline-passengers CSV dataset and read it into a Pandas DataFrame using parse_dates and index_col to set a proper DatetimeIndex.
3. Explore the dataset to inspect structure, missing values, and summary statistics.
4. Plot the time series to visually analyze trends, seasonality patterns, and potential outliers.
5. Perform a stationarity test by running the Augmented Dickey-Fuller (ADF) test on the series.
6. If the series is non-stationary, convert it to stationary by differencing the series.
7. Re-run the ADF test on the differenced series and visualize the resulting DataFrame.
8. If the series remains non-stationary, apply second-order differencing ($d = 2$) or log transformations before differencing.
9. Partition the original time series dataset into training and testing subsets.
10. Fit an ARIMA model on the training set using the appropriate differencing order (d) obtained from step 8.
11. Generate forecasts on the test dataset period using the trained ARIMA model.
12. Compute evaluation metrics: Mean Squared Error (MSE), Mean Absolute Error (MAE), and Root Mean Squared Error (RMSE) on the predictions.
13. Plot the training set, test set, and forecasted predictions together on a single graph for comparison.

1. Download the Airline Passengers Dataset

```
#Exp_No_6b
#Sanjay_S_241801246
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv"
df = pd.read_csv(url)
print(df.head())
```

```
     Month  Passengers
0  1949-01         112
1  1949-02         118
2  1949-03         132
3  1949-04         129
4  1949-05         121
```

2. Read the CSV with parse_dates and index_col

```
#Exp_No_6b
#Sanjay_S_241801246
df = pd.read_csv(
    url,
    parse_dates=["Month"],
    index_col="Month"
)
print(df.head())
print(df.index)
```

```
            Passengers
Month
1949-01-01         112
1949-02-01         118
1949-03-01         132
1949-04-01         129
1949-05-01         121
DatetimeIndex(['1949-01-01', '1949-02-01', '1949-03-01', '1949-04-01',
               '1949-05-01', '1949-06-01', '1949-07-01', '1949-08-01',
               '1949-09-01', '1949-10-01',
               ...
               '1960-03-01', '1960-04-01', '1960-05-01', '1960-06-01',
               '1960-07-01', '1960-08-01', '1960-09-01', '1960-10-01',
               '1960-11-01', '1960-12-01'],
              dtype='datetime64[ns]', name='Month', length=144, freq=None)
```

3. Explore the Dataset

```
#Exp_No_6b
#Sanjay_S_241801246
print("Dataset Shape:", df.shape)
```

```
Dataset Shape: (144, 1)
```

```
print("\nDataset Information:")
df.info()
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 144 entries, 1949-01-01 to 1960-12-01
Data columns (total 1 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Passengers  144 non-null    int64
dtypes: int64(1)
memory usage: 2.2 KB
```

```
print("\nStatistical Summary:")
print(df.describe())
```

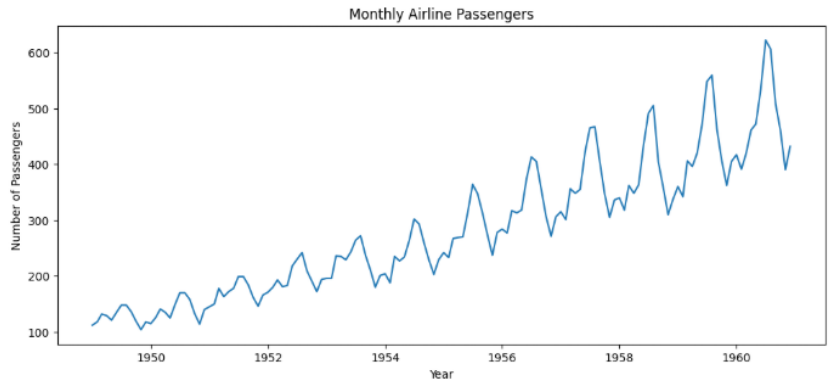
```
Statistical Summary:
       Passengers
count  144.000000
mean   280.298611
std    119.966317
min    104.000000
25%    180.000000
50%    265.500000
75%    360.500000
max    622.000000
```

```
print("\nMissing Values:")
print(df.isnull().sum())
```

```
Missing Values:
Passengers    0
dtype: int64
```

4. Plot the Series to Inspect Trend, Seasonality, and Outliers

```
#Exp_No_6b
#Sanjay_S_241801246
plt.figure(figsize=(12, 5))
plt.plot(df["Passengers"])
plt.title("Monthly Airline Passengers")
plt.xlabel("Year")
plt.ylabel("Number of Passengers")
plt.show()
```



5. Check Stationarity Using the Augmented Dickey-Fuller (ADF) Test

```
#Exp_No_6b
#Sanjay_S_241801246
from statsmodels.tsa.stattools import adfuller
result = adfuller(df["Passengers"])
print("ADF Statistic:", result[0])
print("p-value:", result[1])
if result[1] <= 0.05:
    print("The series is stationary.")
else:
    print("The series is non-stationary.")
```

```
ADF Statistic: 0.8153688792060498
p-value: 0.991880243437641
The series is non-stationary.
/tmp/ipykernel_2584/3796989905.py:2: FutureWarning: adfuller currently returns a plain tuple whose length depends on the store and autolag arguments. In release 0.16 or after July 2027, whi
  result = adfuller(df["Passengers"])
```

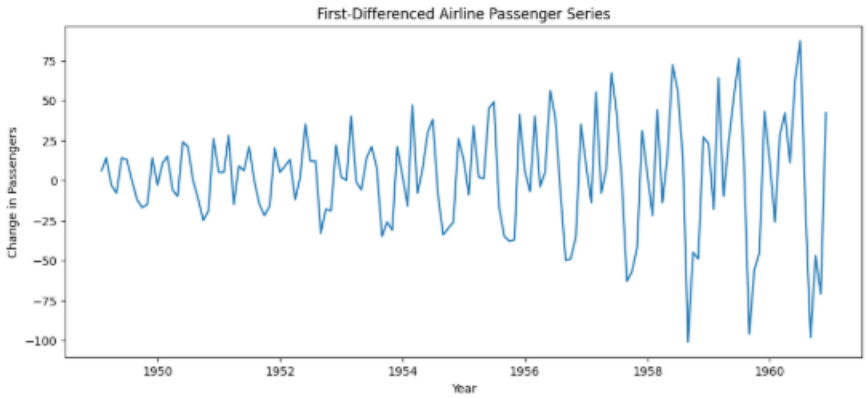
6. Convert the Non-Stationary Series into a Stationary Series Using Differencing

```
#Exp_No_6b
#Sanjay_S_241801246
df_diff1 = df["Passengers"].diff().dropna()
print(df_diff1.head())
result_diff1 = adfuller(df_diff1)
print("ADF Statistic:", result_diff1[0])
print("p-value:", result_diff1[1])
if result_diff1[1] <= 0.05:
    print("The first-differenced series is stationary.")
else:
    print("The first-differenced series is non-stationary.")
```

```
Month
1949-02-01     6.0
1949-03-01    14.0
1949-04-01    -3.0
1949-05-01    -8.0
1949-06-01    14.0
Name: Passengers, dtype: float64
ADF Statistic: -2.8292668241700047
p-value: 0.05421329028382478
The first-differenced series is non-stationary.
/tmp/ipykernel_2584/497467305.py:3: FutureWarning: adfuller currently returns a plain tuple whose length depends on the store and autolag arguments. In release 0.16 or after
  result_diff1 = adfuller(df_diff1)
```

7. Re-run ADF on the Differenced Series and Visualize It

```
#Exp_No_6b
#Sanjay_S_241801246
plt.figure(figsize=(12, 5))
plt.plot(df_diff1)
plt.title("First-Differenced Airline Passenger Series")
plt.xlabel("Year")
plt.ylabel("Change in Passengers")
plt.show()
result = adfuller(df_diff1)
print("ADF Statistic:", result[0])
print("p-value:", result[1])
```

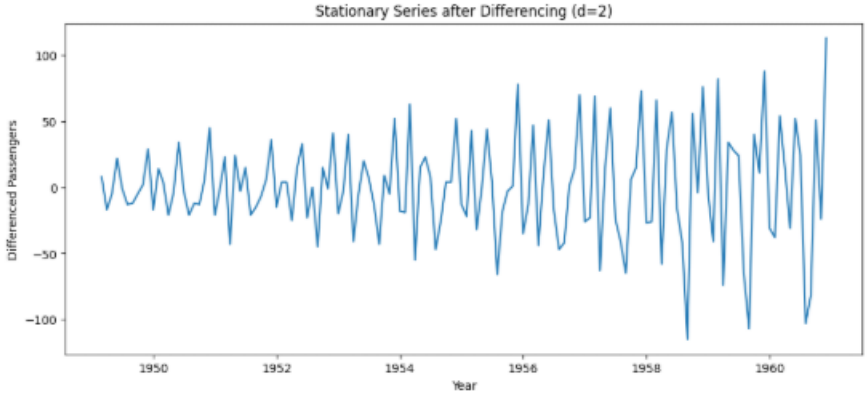



```
ADF Statistic: -2.8292668241700047
p-value: 0.05421329028382478
/tmp/ipykernel_2584/901687768.py:7: FutureWarning: adfuller currently returns a plain tuple whose length depends on the store and autolag arguments. In release 0.16 or after
  result = adfuller(df_diff1)
```

8. Apply Second Differencing if the Series Is Still Non-Stationary

```
#Exp_No_6b
#Sanjay_S_241801246
df_diff2 = df["Passengers"].diff().diff().dropna()
result_diff2 = adfuller(df_diff2)
print("Second-Difference ADF Statistic:", result_diff2[0])
print("Second-Difference p-value:", result_diff2[1])
if result_diff1[1] <= 0.05:
    d = 1
    stationary_series = df_diff1
elif result_diff2[1] <= 0.05:
    d = 2
    stationary_series = df_diff2
else:
    d = 2
    stationary_series = df_diff2
    print("Series may require further transformation.")
print("Selected differencing order (d):", d)
plt.figure(figsize=(12, 5))
plt.plot(stationary_series)
plt.title(f"Stationary Series after Differencing (d={d})")
plt.xlabel("Year")
plt.ylabel("Differenced Passengers")
plt.show()
```

```
Second-Difference ADF Statistic: -16.384231542468513
Second-Difference p-value: 2.7328918500142026e-29
Selected differencing order (d): 2
/tmp/ipykernel_2584/1860364274.py:2: FutureWarning: adfuller currently returns a plain tuple whose length depends on the store and autolag arguments.
  result_diff2 = adfuller(df_diff2)
```



9. Split the Dataset into Training and Testing Sets

```
#Exp_No_6b
#Sanjay_S_241801246
passengers = df["Passengers"]
train_size = int(len(passengers) * 0.8)
train = passengers.iloc[:train_size]
test = passengers.iloc[train_size:]
print("Training Data:", len(train))
print("Testing Data:", len(test))
```

```
Training Data: 115
Testing Data: 29
```

10. Fit the ARIMA Model on the Training Set

```
#Exp_No_6b
#Sanjay_S_241801246
from statsmodels.tsa.arima.model import ARIMA
model = ARIMA(train, order=(5, d, 0))
model_fit = model.fit()
print(model_fit.summary())
```

```
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: No frequency information was provided, so inferred frequency MS will be used.
  self._init_dates(dates, freq)
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: No frequency information was provided, so inferred frequency MS will be used.
  self._init_dates(dates, freq)
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: No frequency information was provided, so inferred frequency MS will be used.
  self._init_dates(dates, freq)
                               SARIMAX Results
==============================================================================
Dep. Variable:              Passengers   No. Observations:                  115
Model:                 ARIMA(5, 2, 0)   Log Likelihood                -531.612
Date:                Sun, 11 Oct 2026   AIC                           1075.223
Time:                        06:08:08   BIC                           1091.587
Sample:                    01-01-1949   HQIC                          1081.864
                         - 07-01-1958
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1         -0.4046      0.106     -3.811      0.000      -0.613      -0.197
ar.L2         -0.5093      0.120     -4.249      0.000      -0.744      -0.274
ar.L3         -0.2456      0.081     -3.027      0.002      -0.405      -0.087
ar.L4         -0.4367      0.089     -4.885      0.000      -0.612      -0.261
ar.L5         -0.1433      0.093     -1.537      0.124      -0.326       0.039
sigma2       707.6066    104.704      6.758      0.000     502.391     912.822
===================================================================================
Ljung-Box (L1) (Q):                   0.01   Jarque-Bera (JB):                 0.33
Prob(Q):                              0.90   Prob(JB):                         0.85
Heteroskedasticity (H):               5.47   Skew:                             0.07
Prob(H) (two-sided):                  0.00   Kurtosis:                         3.22
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```

11. Perform Forecasts on the Test Dataset

```
#Exp_No_6b
#Sanjay_S_241801246
predictions = model_fit.forecast(steps=len(test))
predictions.index = test.index
print("Forecasted Passenger Numbers:")
print(predictions)
```

```
Forecasted Passenger Numbers:
Month
1958-08-01     533.189557
1958-09-01     570.766204
1958-10-01     592.124375
1958-11-01     624.603396
1958-12-01     670.300205
1959-01-01     712.961841
1959-02-01     755.132070
1959-03-01     793.268943
1959-04-01     826.667362
1959-05-01     863.588943
1959-06-01     903.138546
1959-07-01     942.825916
1959-08-01     982.901158
1959-09-01    1021.244474
1959-10-01    1058.404551
1959-11-01    1096.393393
1959-12-01    1134.885764
1960-01-01    1173.743622
1960-02-01    1212.758558
1960-03-01    1251.207798
1960-04-01    1289.377490
1960-05-01    1327.678056
1960-06-01    1366.085996
1960-07-01    1404.677013
1960-08-01    1443.310284
1960-09-01    1481.789759
1960-10-01    1520.199332
1960-11-01    1558.609802
1960-12-01    1597.048591
Name: predicted_mean, dtype: float64
```

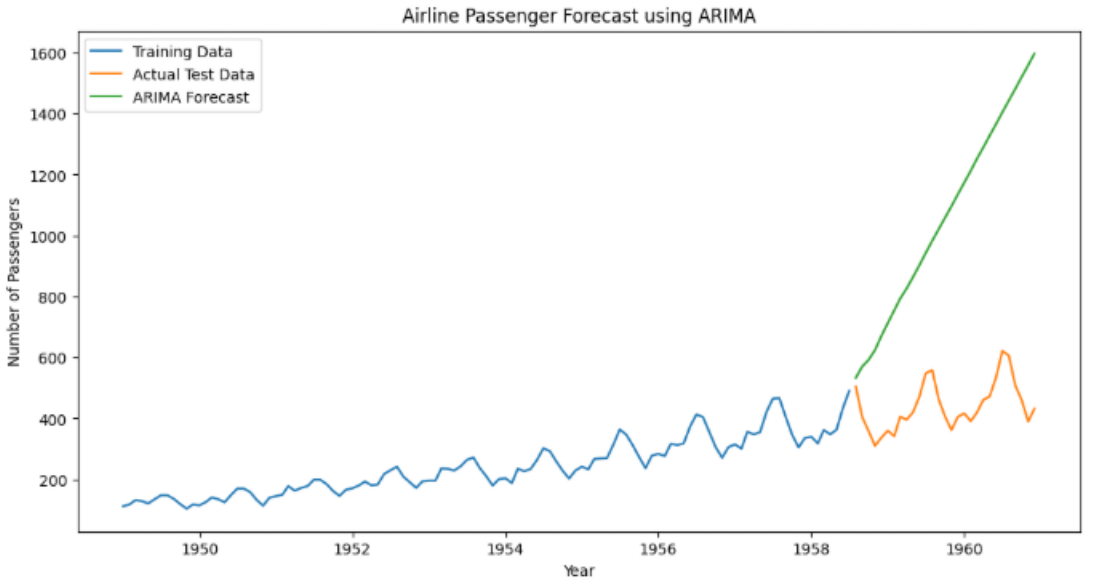
12. Compute MSE, MAE, and RMSE

```
#Exp_No_6b
#Sanjay_S_241801246
from sklearn.metrics import mean_squared_error, mean_absolute_error
mse = mean_squared_error(test, predictions)
mae = mean_absolute_error(test, predictions)
rmse = np.sqrt(mse)
print("Mean Squared Error (MSE):", mse)
print("Mean Absolute Error (MAE):", mae)
print("Root Mean Squared Error (RMSE):", rmse)
```

```
Mean Squared Error (MSE): 469187.4575741138
Mean Absolute Error (MAE): 618.6166551413303
Root Mean Squared Error (RMSE): 684.972596221275
```

13. Plot Training Data, Testing Data, and Predictions in a Single Graph

```
#Exp_No_6b
#Sanjay_S_241801246
plt.figure(figsize=(12, 6))
plt.plot(train.index, train, label="Training Data")
plt.plot(test.index, test, label="Actual Test Data")
plt.plot(predictions.index, predictions, label="ARIMA Forecast")
plt.title("Airline Passenger Forecast using ARIMA")
plt.xlabel("Year")
plt.ylabel("Number of Passengers")
plt.legend()
plt.show()
```



RESULT: Time series data analysis and forecasting on the Airline Passengers dataset were successfully implemented. The stationarity of the series was evaluated, appropriate differencing was applied, and an ARIMA model was trained, forecasted, and evaluated using MSE, MAE, and RMSE metrics alongside a visual performance plot.

Ex7: Model Planning and Building - Clustering

AIM: To implement the K-Means clustering algorithm on the Mall Customer Segmentation dataset for customer segmentation by exploring data, standardizing key features, identifying the optimal number of clusters using the Elbow Method, visualizing cluster distributions and centroids, and analyzing cluster statistics.

ALGORITHM:

1. Import necessary Python libraries (pandas, numpy, matplotlib.pyplot, seaborn, StandardScaler from sklearn.preprocessing, and KMeans from sklearn.cluster).
2. Load the Mall Customer Segmentation Dataset from Kaggle into a Pandas DataFrame.
3. Explore the dataset by displaying the first and last five records, checking shape and data types, identifying missing values and duplicate records, and displaying summary statistics.
4. Select the target features for clustering: Annual Income (k$) and Spending Score (1-100).
5. Standardize the selected feature columns using StandardScaler.
6. Visualize the unscaled/scaled features using Scatter Plots and Pair Plots.
7. Determine the optimal number of clusters (K) using the Elbow Method by plotting the Elbow Curve (within-cluster sum of squares vs. K).
8. Train the K-Means Clustering model using the optimal value of K.
9. Assign cluster labels to each customer, append the labels as a new column in the DataFrame, and save the resulting dataset as a file.
10. Visualize customer clusters on a scatter plot (X-axis: Annual Income, Y-axis: Spending Score) using unique colors for each cluster, highlighting cluster centroids, and displaying centroid coordinates.
11. Perform cluster analysis by calculating customer counts per cluster and computing the average Annual Income and Spending Score for each segment.

EX7 – Model Planning and Building: Clustering

1. Load the Mall Customer Segmentation Dataset into a DataFrame

```
#Exp_No_7
#Sanjay_S_241801246
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
df = pd.read_csv("Mall_Customers.csv")
print(df.head())
```

```
   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40
```

2. Explore the Dataset

2.1 Display the First and Last Five Records

```
#Exp_No_7
#Sanjay_S_241801246
print("First Five Records:")
print(df.head())
print("\nLast Five Records:")
print(df.tail())
```

```
First Five Records:
   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40

Last Five Records:
     CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)
195         196  Female   35                 120                      79
196         197  Female   45                 126                      28
197         198    Male   32                 126                      74
198         199    Male   32                 137                      18
199         200    Male   30                 137                      83
```

2.2 Check Dataset Shape and Data Types

```
#Exp_No_7
#Sanjay_S_241801246
print("Dataset Shape:", df.shape)
```

```
Dataset Shape: (200, 5)
```

```
print("\nData Types:")
print(df.dtypes)
```

```
Data Types:
CustomerID                int64
Gender                   object
Age                       int64
Annual Income (k$)        int64
Spending Score (1-100)    int64
dtype: object
```

```
print("\nDataset Information:")
df.info()
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   CustomerID              200 non-null    int64
 1   Gender                  200 non-null    object
 2   Age                     200 non-null    int64
 3   Annual Income (k$)      200 non-null    int64
 4   Spending Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
```

2.3 Identify Missing and Duplicate Records

```
print("Missing Values:")
print(df.isnull().sum())
print("\nDuplicate Records:", df.duplicated().sum())
```

```
Missing Values:
CustomerID              0
Gender                  0
Age                     0
Annual Income (k$)      0
Spending Score (1-100)  0
dtype: int64

Duplicate Records: 0
```

2.4 Display Summary Statistics

```
print(df.describe(include="all"))
```

```
        CustomerID  Gender         Age  Annual Income (k$)  \
count   200.000000     200  200.000000          200.000000
unique         NaN       2         NaN                 NaN
top            NaN  Female         NaN                 NaN
freq           NaN     112         NaN                 NaN
mean    100.500000     NaN   38.850000           60.560000
std      57.879185     NaN   13.969007           26.264721
min       1.000000     NaN   18.000000           15.000000
25%      50.750000     NaN   28.750000           41.500000
50%     100.500000     NaN   36.000000           61.500000
75%     150.250000     NaN   49.000000           78.000000
max     200.000000     NaN   70.000000          137.000000

        Spending Score (1-100)
count               200.000000
unique                     NaN
top                        NaN
freq                       NaN
mean                 50.200000
std                  25.823522
min                   1.000000
25%                  34.750000
50%                  50.000000
75%                  73.000000
max                  99.000000
```

3. Select Important Features for Clustering

```
#Exp_No_7
#Sanjay_S_241801246
X = df[["Annual Income (k$)", "Spending Score (1-100)"]]
print(X.head())
```

```
   Annual Income (k$)  Spending Score (1-100)
0                  15                      39
1                  15                      81
2                  16                       6
3                  16                      77
4                  17                      40
```

4. Standardize the Selected Features Using StandardScaler

```
#Exp_No_7
#Sanjay_S_241801246
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print("Standardized Features:")
print(X_scaled[:5])
```

```
Standardized Features:
[[-1.73899919 -0.43480148]
 [-1.73899919  1.19570407]
 [-1.70082976 -1.71591298]
 [-1.70082976  1.04041783]
 [-1.66266033 -0.39597992]]
```

5. Visualize the Selected Features

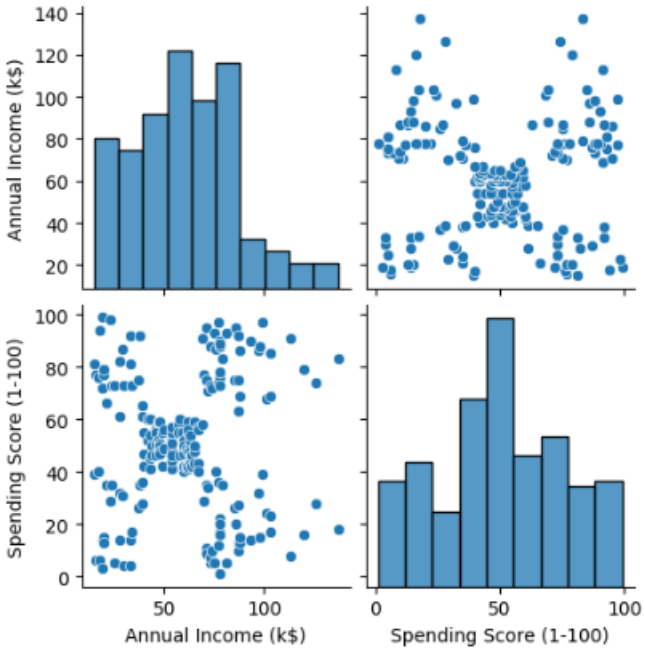
5.1 Scatter Plot

```
#Exp_No_7
#Sanjay_S_241801246
plt.figure(figsize=(8, 5))
plt.scatter(X["Annual Income (k$)"], X["Spending Score (1-100)"])
plt.title("Annual Income vs Spending Score")
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.show()
```



5.2 Pair Plot

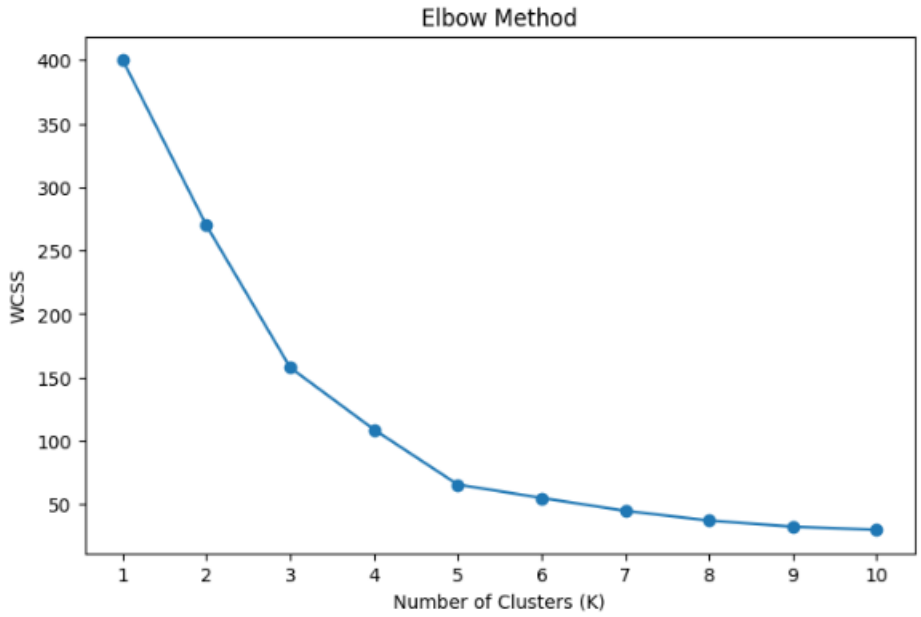
```
#Exp_No_7
#Sanjay_S_241801246
sns.pairplot(
    df[["Annual Income (k$)", "Spending Score (1-100)"]]
)
plt.show()
```



Model Planning and Building

6. Determine the Optimal Number of Clusters Using the Elbow Method

```
#Exp_No_7
#Sanjay_S_241801246
from sklearn.cluster import KMeans
wcss = []
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)
plt.figure(figsize=(8, 5))
plt.plot(range(1, 11), wcss, marker="o")
plt.title("Elbow Method")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.xticks(range(1, 11))
plt.show()
```



7. Train K-Means, Assign Cluster Labels, and Save the Dataset

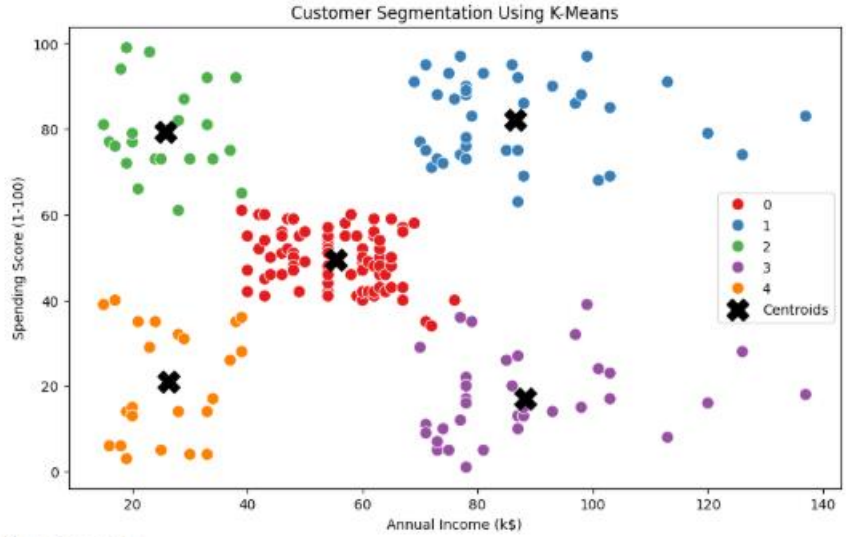
```
#Exp_No_7
#Sanjay_S_241801246
kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)
df["Cluster"] = kmeans.fit_predict(X_scaled)
print(df.head())
df.to_csv("Mall_Customers_Clustered.csv", index=False)
print("Clustered dataset saved successfully.")
```

```
   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)  \
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40

   Cluster
0        4
1        2
2        4
3        2
4        4
Clustered dataset saved successfully.
```

8. Visualize Customer Clusters and Display Centroids

```
#Exp_No_7
#Sanjay_S_241801246
centroids = scaler.inverse_transform(kmeans.cluster_centers_)
plt.figure(figsize=(10, 6))
sns.scatterplot(
    data=df,
    x="Annual Income (k$)",
    y="Spending Score (1-100)",
    hue="Cluster",
    palette="Set1",
    s=80
)
plt.scatter(
    centroids[:, 0],
    centroids[:, 1],
    marker="X",
    s=250,
    color="black",
    label="Centroids"
)
plt.title("Customer Segmentation Using K-Means")
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.legend()
plt.show()
print("Cluster Centroids:")
print(pd.DataFrame(
    centroids,
    columns=["Annual Income (k$)", "Spending Score (1-100)"]
))
```

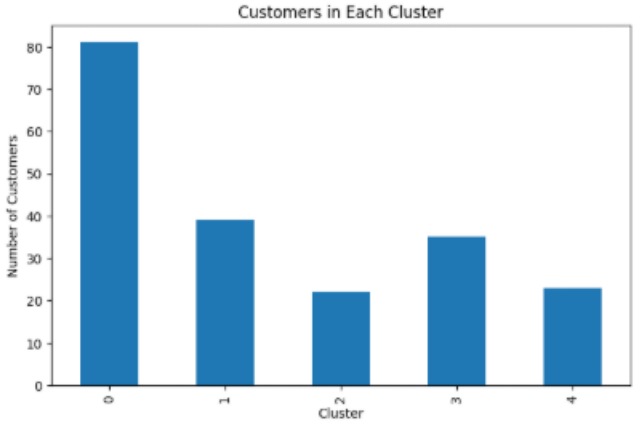



```
Cluster Centroids:
   Annual Income (k$)  Spending Score (1-100)
0           55.296296               49.518519
1           86.538462               82.128205
2           25.727273               79.363636
3           88.200000               17.114286
4           26.304348               20.913043
```

9. Calculate the Number of Customers in Each Cluster

```
#Exp_No_7
#Sanjay_S_241801246
cluster_counts = df["Cluster"].value_counts().sort_index()
print("Number of Customers in Each Cluster:")
print(cluster_counts)
cluster_counts.plot(kind="bar", figsize=(8, 5))
plt.title("Customers in Each Cluster")
plt.xlabel("Cluster")
plt.ylabel("Number of Customers")
plt.show()
```

```
Number of Customers in Each Cluster:
Cluster
0    81
1    39
2    22
3    35
4    23
Name: count, dtype: int64
```



10. Compare Average Annual Income and Spending Score for Each Cluster

```
#Exp_No_7
#Sanjay_S_241801246
cluster_analysis = df.groupby("Cluster")[
    ["Annual Income (k$)", "Spending Score (1-100)"]
].mean()
print("Average Income and Spending Score by Cluster:")
print(cluster_analysis)
```

```
Average Income and Spending Score by Cluster:
         Annual Income (k$)  Spending Score (1-100)
Cluster
0                 55.296296               49.518519
1                 86.538462               82.128205
2                 25.727273               79.363636
3                 88.200000               17.114286
4                 26.304348               20.913043
```

RESULT: The K-Means clustering algorithm was successfully implemented for customer segmentation on the Mall Customer Segmentation dataset. Feature standardization, optimal cluster selection via the Elbow Method, model fitting, centroid visualization, and cluster summary statistics were completed.

Ex8: Interactive Dashboards with Tableau

AIM: To design and build an interactive data visualization dashboard in Tableau using the Sample – Superstore dataset by creating bar, line, and scatter plot worksheets, organizing them on a dashboard, and configuring interactive filters and actions.

ALGORITHM:

1. Open Tableau and load the Sample – Superstore dataset from Kaggle, then explore the underlying data fields.
2. Create **Worksheet 1 – Bar Chart (Bar_Sales_by_Category)**:
    - Drag Category to Columns and SUM(Sales) to Rows.
    - Sort the bars in descending order.
    - Drag Region to Color and turn on "Show Mark Labels".
    - (Optional) Drag Region to Filters and select "Show Filter".
3. Create **Worksheet 2 – Line Chart (Line_Sales_Trend)**:
    - Drag Order Date to Columns and set it to continuous Month.
    - Drag SUM(Sales) to Rows and Region to Color.
    - Format the axis as Currency.
4. Create **Worksheet 3 – Scatter Plot (Scatter_Profit_vs_Sales)**:
    - Drag SUM(Sales) to Columns and SUM(Profit) to Rows.
    - Drag Sub-Category to Detail and Category to Color.
    - (Optional) Drag Segment to Shape and add reference lines for average values on both axes.
5. **Build the Dashboard (Sales Dashboard)**:
    - Create a new dashboard and set dimensions (e.g., 1200×800).
    - Add the title "Sales and Profit Dashboard" and arrange worksheets using layout containers: Line chart at top, Bar chart at bottom-left, and Scatter plot at bottom-right.
6. **Add Interactivity**:
    - Display filters for fields such as Region, Category, or Year.

- Configure Dashboard Filter Actions using the Bar chart as the source sheet and the Line and Scatter plots as target sheets triggered on selection.
- (Optional) Add Highlight Actions across charts.

7. **Publish/Export**: Save the finished interactive dashboard to Tableau Public or export as an Image/PDF.



RESULT: An interactive Tableau dashboard titled "Sales and Profit Dashboard" was successfully created using the Sample – Superstore dataset. The integrated worksheets, dynamic filters, and selection actions allow seamless interactive analysis of sales trends, regional performance, and product profitability